```
3  import math
4
5  class Circle(GeometricObject):
6      def __init__(self, radius):
7          super().__init__()
8          self.setRadius(radius)
9
10     def getRadius(self):
11         return self.__radius
12
13     def setRadius(self, radius):
14         if radius >= 0:
15             self.__radius = radius
16         else:
17             raise InvalidRadiusException(radius)
18
19     def getArea(self):
20         return self.__radius * self.__radius * math.pi
21
22     def getDiameter(self):
23         return 2 * self.__radius
24
25     def getPerimeter(self):
26         return 2 * self.__radius * math.pi
27
28     def printCircle(self):
29         print(self.__str__(), "radius:", self.__radius)
```

如果半径是负数，setRadius 方法抛出一个 InvalidRadiusException 异常（第 17 行）。程序清单 13-16 给出了一个使用程序清单 13-15 中的新 Circle 类创建圆对象的测试程序。

程序清单 13-16 TestCircleWithCustomException.py

```
1  from CircleWithCustomException import Circle
2  from InvalidRadiusException import InvalidRadiusException
3
4  try:
5      c1 = Circle(5)
6      print("c1's area is", c1.getArea())
7      c2 = Circle(-5)
8      print("c2's area is", c2.getArea())
9      c3 = Circle(0)
10     print("c3's area is", c3.getArea())
11 except InvalidRadiusException as ex:
12     print("The radius", ex.radius, "is invalid")
13 except Exception:
14     print("Something is wrong")
```

```
c1's area is 78.53981633974483
The radius -5 is invalid
```

当使用一个负数半径来创建一个 Circle 对象时（第 7 行），一个 InvalidRadiusException 异常就被抛出。这个异常被第 11 ～ 12 行的 except 子句捕获。

except 块中异常顺序的指定是很重要的，因为 Python 是按这个顺序来寻找异常处理器的。如果一个父类类型异常的 except 块出现在子类类型异常的 except 块之前，那么这个子类类型异常的 except 块将永远不会被执行。例如，如下编写代码是错误的。

```
try:
    ....
except Exception:
```

```
    print("Something is wrong")
except InvalidRadiusException:
    print("Invalid radius")
```

检查点

13.18　假设下面 try-except 块中的 statement2 子句出现一个异常：

```
try:
    statement1
    statement2
    statement3
except Exception1:
    # Handle exception 1
except Exception2:
    # Handle exception 2

statement4
```

回答下面的问题：

- statement3 会被执行吗？
- 如果异常未被捕获，那么 statement4 会被执行吗？
- 如果异常在 except 块中被捕获，那么 statement4 会被执行吗？

13.19　运行下面的程序时显示什么？

```
try:
    list = 10 * [0]
    x = list[10]
    print("Done ")
except IndexError:
    print("Index out of bound")
```

13.20　运行下面的程序时显示什么？

```
def main():
    try:
        f()
        print("After the function call")
    except ZeroDivisionError:
        print("Divided by zero!")
    except:
        print("Exception")

def f():
    print(1 / 0)

main() # Call the main function
```

13.21　运行下面的程序时显示什么？

```
def main():
    try:
        f()
        print("After the function call")
    except IndexError:
        print("Index out of bound")
    except:
        print("Exception in main")

def f():
    try:
        s ="abc"
        print(s[3])
```

```
    except ZeroDivisionError:
        print("Divided by zero!")

main() # Call the main function
```

13.22　假设下面的语句中 statement2 子句引起了一个异常：

```
try:
    statement1
    statement2
    statement3
except Exception1:
    # Handle exception
except Exception2:
    # Handle exception
except Exception3:
    # Handle exception
finally:
    statement4

statement5
```

回答下面的问题：

- 如果异常未被捕获，那么 statement5 会被执行吗？
- 如果异常类型是 Exception3，那么 statement4 和 statement5 会被执行吗？

13.23　如何在函数中抛出一个异常？

13.24　使用异常处理的优势是什么？

13.25　运行下面的程序时显示什么？

```
try:
    lst = 10 * [0]
    x = lst[9]
    print("Done")
except IndexError:
    print("Index out of bound")
else:
    print("Nothing is wrong")
finally:
    print("Finally we are here")

print("Continue")
```

13.26　运行下面的程序时显示什么？

```
try:
    lst = 10 * [0]
    x = lst[10]
    print("Done ")
except IndexError:
    print("Index out of bound")
else:
    print("Nothing is wrong")
finally:
    print("Finally we are here")

print("Continue")
```

13.27　下面代码错在哪里？

```
try:
   # Some code here
   ...
```

```
except ArithmeticError:
    print("ArithmeticError")
except ZeroDivisionError:
    print("ZeroDivisionError")

print("Continue")
```

13.28　如何定义一个自定制异常类？

13.10　使用 Pickling 进行二进制 IO

关键点：为了使用 Pickling 进行二进制 IO，使用模式 rb 和 wb 打开一个文件以进行二进制读写并调用 pickle 模块中的 dump 和 load 函数来读写数据。

可以向一个文件写入字符串和数字。可以向文件直接写入像列表这样的任何一个对象吗？答案是肯定的。这需要二进制 IO。Python 中有很多方法进行二进制 IO。本节介绍如何使用 pickle 模块中的 dump 和 load 函数进行二进制 IO。

Python 的 pickle 模块使用强大且有效的算法来序列化和反序列化对象。*序列化*是指将一个对象转换为一个能够存储在一个文件中或在网络上进行传输的字节流的过程。*反序列化*指的是相反的过程，它是从字节流中提取出对象的过程。序列化 / 反序列化在 Python 中也称为*浸渍 / 去渍*或*卸载 / 加载*对象。

13.10.1　卸载和加载对象

众所周知，Python 中的所有数据都是对象。pickle 模块使我们能够通过使用 dump 和 load 函数来写入 / 读取任何数据。程序清单 13-17 演示这些函数。

程序清单 13-17　BinaryIODemo.py

```
1  import pickle
2
3  def main():
4      # Open file for writing binary
5      outfile = open("pickle.dat", "wb")
6      pickle.dump(45, outfile)
7      pickle.dump(56.6, outfile)
8      pickle.dump("Programming is fun", outfile)
9      pickle.dump([1, 2, 3, 4], outfile)
10     outfile.close() # Close the output file
11
12     # Open file for reading binary
13     infile = open("pickle.dat", "rb")
14     print(pickle.load(infile))
15     print(pickle.load(infile))
16     print(pickle.load(infile))
17     print(pickle.load(infile))
18     infile.close() # Close the input file
19
20 main() # Call the main function
```

```
45
56.6
Programming is fun
[1, 2, 3, 4]
```

为了使用 pickle，需要导入 pickle 模块（第 1 行）。为了向一个文件写入对象，使用 wb 模式打开这个文件以进行二进制写入（第 5 行），使用 dump(object) 方法将对象写入到文件

中（第 6 ～ 9 行）。这个方法将对象序列化为一个字节流并将它们存入文件中。

这个程序关闭文件（第 10 行）并打开它来读取二进制数据（第 13 行）。使用 load 方法来读取对象（第 14 ～ 17 行）。这个方法读取一个字节流并将它们反序列化为一个对象。

13.10.2　检测文件末尾

如果不知道文件中有多少对象，那么如何读取文件的所有对象？可以通过使用 load 函数重复读取一个对象直到函数抛出一个 EOFError 异常（文件末尾）。当抛出这个异常时，捕获并处理它以结束文件读取过程。

程序清单 13-18 中的程序通过使用对象 IO 将未指定数目的整数对象存入一个文件，然后将它们从文件中读取出来。

程序清单 13-18　DetectEndOfFile.py

```
1  import pickle
2
3  def main():
4      # Open file for writing binary
5      outfile = open("numbers.dat", "wb")
6
7      data = eval(input("Enter an integer (the input exits " +
8          "if the input is 0): "))
9      while data != 0:
10         pickle.dump(data, outfile)
11         data = eval(input("Enter an integer (the input exits " +
12             "if the input is 0): "))
13
14     outfile.close() # Close the output file
15
16     # Open file for reading binary
17     infile = open("numbers.dat", "rb")
18
19     end_of_file = False
20     while not end_of_file:
21         try:
22             print(pickle.load(infile), end = " ")
23         except EOFError:
24             end_of_file = True
25
26     infile.close() # Close the input file
27
28     print("\nAll objects are read")
29
30 main() # Call the main function
```

```
Enter an integer (the input exits if the input is 0): 4 ↵Enter
Enter an integer (the input exits if the input is 0): 5 ↵Enter
Enter an integer (the input exits if the input is 0): 7 ↵Enter
Enter an integer (the input exits if the input is 0): 9 ↵Enter
Enter an integer (the input exits if the input is 0): 0 ↵Enter
4 5 7 9
All objects are read
```

这个程序打开文件以写入二进制数据（第 5 行），并且不断提示用户输入一个整数并使用 dump 函数将它存入文件（第 10 行）直到输入的整数为 0。

关闭这个文件（第 14 行）并重新打开它以读取二进制数据（第 17 行）。程序利用 while 循环中的 load 函数重复读取一个对象直到出现一个 EOFError 异常。当 EOFError 异常出现时，end_of_file 被设置为 True 以终止 while 循环（第 20 行）。

如实例输出所示，用户输入四个整数并存入文件，然后从文件中重新读取它们并在控制台上显示。

检查点

13.29　如何打开一个文件进行对象的写入和读取？

13.30　如何调用函数来读写一个对象？

13.31　如果将程序清单 13-18 中的第 20 ～ 24 行的代码用下面的代码替换，将会出现什么错误？

```
while not end_of_file:
    try:
       print(pickle.load(infile), end = " ")
    except EOFError:
       end_of_file = True
    finally:
       infile.close() # Close the input file
```

13.32　能否用下面代码替换程序清单 13-18 中的第 20 ～ 24 行代码？

```
try:
    while not end_of_file:
        print(pickle.load(infile), end = " ")
except EOFError:
    print("\nAll objects are read")
finally:
    infile.close() # Close the input file
```

13.11　实例研究：地址簿

关键点：这个例子的问题是用二进制 IO 创建一个地址簿。

现在，让我们使用对象 IO 来创建一个有用的工程来存储和查看地址簿。

程序的用户接口如图 13-11 所示。“Add”按钮将存储一个新地址到文件末尾。“First”、“Next”、“Previous”和“Last”按钮分别用来获取文件的第一个、下一个、上一个和最后一个地址。

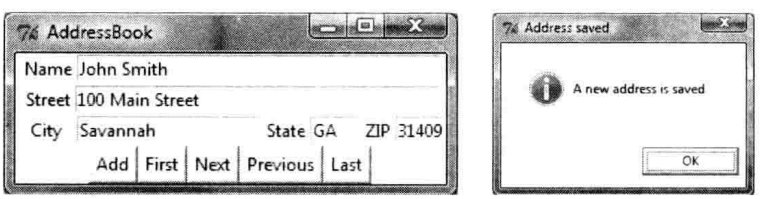


图 13-11　AddressBook 通过一个文件存储和获取地址

我们将定义一个名为 Address 的类来代表一个地址，使用一个列表来存储所有地址。当单击“Add”按钮时，程序创建一个 Address 对象，通过用户输入它的姓名、街区、城市、国家和邮编给这个对象赋值，将这个对象追加到列表中，并且使用二进制 IO 将这个列表存储到文件中。假设文件名为 address.dat。

当运行程序时，首先将从文件中读取这个列表并在用户接口上显示列表的第一个地址。如果文件为空，那么显示空输入框。这个程序在程序清单 13-19 中给出。

程序清单 13-19 AddressBook.py

```
import pickle
import os.path
from tkinter import * # Import all definitions from tkinter
import tkinter.messagebox

class Address:
    def __init__(self, name, street, city, state, zip):
        self.name = name
        self.street = street
        self.city = city
        self.state = state
        self.zip = zip

class AddressBook:
    def __init__(self):
        window = Tk() # Create a window
        window.title("AddressBook") # Set title

        self.nameVar = StringVar()
        self.streetVar = StringVar()
        self.cityVar = StringVar()
        self.stateVar = StringVar()
        self.zipVar = StringVar()

        frame1 = Frame(window)
        frame1.pack()
        Label(frame1, text = "Name").grid(row = 1,
            column = 1, sticky = W)
        Entry(frame1, textvariable = self.nameVar,
            width = 40).grid(row = 1, column = 2)

        frame2 = Frame(window)
        frame2.pack()
        Label(frame2, text = "Street").grid(row = 1,
            column = 1, sticky = W)
        Entry(frame2, textvariable = self.streetVar,
            width = 40).grid(row = 1, column = 2)

        frame3 = Frame(window)
        frame3.pack()
        Label(frame3, text = "City", width = 5).grid(row = 1,
            column = 1, sticky = W)
        Entry(frame3,
            textvariable = self.cityVar).grid(row = 1, column = 2)
        Label(frame3, text = "State").grid(row = 1,
            column = 3, sticky = W)
        Entry(frame3, textvariable = self.stateVar,
            width = 5).grid(row = 1, column = 4)
        Label(frame3, text = "ZIP").grid(row = 1,
            column = 5, sticky = W)
        Entry(frame3, textvariable = self.zipVar,
            width = 5).grid(row = 1, column = 6)

        frame4 = Frame(window)
        frame4.pack()
        Button(frame4, text = "Add",
            command = self.processAdd).grid(row = 1, column = 1)
        btFirst = Button(frame4, text = "First",
            command = self.processFirst).grid(row = 1, column = 2)
        btNext = Button(frame4, text = "Next",
            command = self.processNext).grid(row = 1, column = 3)
        btPrevious = Button(frame4, text = "Previous", command =
```

```
63              self.processPrevious).grid(row = 1, column = 4)
64          btLast = Button(frame4, text = "Last",
65              command = self.processLast).grid(row = 1, column = 5)
66
67          self.addressList = self.loadAddress()
68          self.current = 0
69
70          if len(self.addressList) > 0:
71              self.setAddress()
72
73          window.mainloop() # Create an event loop
74
75      def saveAddress(self):
76          outfile = open("address.dat", "wb")
77          pickle.dump(self.addressList, outfile)
78          tkinter.messagebox.showinfo(
79              "Address saved", "A new address is saved")
80          outfile.close()
81
82      def loadAddress(self):
83          if not os.path.isfile("address.dat"):
84              return [] # Return an empty list
85
86          try:
87              infile = open("address.dat", "rb")
88              addressList = pickle.load(infile)
89          except EOFError:
90              addressList = []
91
92          infile.close()
93          return addressList
94
95      def processAdd(self):
96          address = Address(self.nameVar.get(),
97              self.streetVar.get(), self.cityVar.get(),
98              self.stateVar.get(), self.zipVar.get())
99          self.addressList.append(address)
100         self.saveAddress()
101
102     def processFirst(self):
103         self.current = 0
104         self.setAddress()
105
106     def processNext(self):
107         if self.current < len(self.addressList) - 1:
108             self.current += 1
109             self.setAddress()
110
111     def processPrevious(self):
112         print("Left as exercise")
113
114     def processLast(self):
115         print("Left as exercise")
116
117     def setAddress(self):
118         self.nameVar.set(self.addressList[self.current].name)
119         self.streetVar.set(self.addressList[self.current].street)
120         self.cityVar.set(self.addressList[self.current].city)
121         self.stateVar.set(self.addressList[self.current].state)
122         self.zipVar.set(self.addressList[self.current].zip)
123
124 AddressBook() # Create GUI
```

Address 类中定义了 _ _init_ _ 方法来创建一个带有名字、街区、城市、国家和邮编的

Address 对象（第 6 ～ 12 行）。

AddressBook 中的 __init__ 方法创建用来显示和处理地址的用户接口（第 25 ～ 65 行）。它从文件读取地址列表（第 67 行），并且将当前列表的地址索引号下表设置为 0（第 68 行）。如果地址列表不为空，程序将显示第一个地址（第 70 ～ 71 行）。

saveAddress 方法向文件写入地址列表（第 77 行），然后显示一个消息框来提醒用户已经添加了新地址（第 78 ～ 79 行）。

loadAddress 方法从文件中读取地址列表（第 88 行）。如果文件不存在，程序将返回一个空列表（第 83 ～ 84 行）。

processAdd 方法使用输入框的值创建一个 Address 对象。它将这个对象追加到列表当中（第 99 行）并调用 saveAddress 方法来将最新更新的列表存到文件中（第 100 行）。

processFirst 方法将 current 重置为 0，它指向地址列表的第一个地址（第 103 行）。然后通过调用 setAddress 方法来将地址输出到输入框内（第 104 行）。

如果 current 不是指向列表中的最后一个地址的话（第 107 行），processNext 方法将 current 移动到指向列表中的下一个地址（第 108 行），并且重置输入框中的地址（第 109 行）。

setAddress 方法为输入框设置地址域（第 117 ～ 122 行）。方法 processPrevious 和 processLast 被留作一个练习题。

关键术语

absolute filename（绝对文件名）
binary file（二进制文件）
deserializing（反序列化）
directory path（目录路径）
file pointer（文件指针）
raw string（原始字符串）
relative filename（相对文件名）
serializing（序列化）
text file（文本文件）
traceback（回溯）

本章总结

1. 可以使用文件对象来从（向）文件读（写）数据。可以打开文件，用 r 模式进行读取，用 w 模式进行写入，用 a 模式进行追加。
2. 可以使用 os.path.isfile(f) 函数来检测一个文件是否存在。
3. Python 中有一个文件类，该类包含了读写数据和关闭文件的方法。
4. 可以使用 read()、readline() 和 readlines() 方法从文件读取数据。
5. 可以使用 write(s) 方法来将一个字符串写入文件。
6. 在文件处理结束后关闭该文件以确保数据被正确保存。
7. 可以像从一个文件中读取数据一样从网页上读取资源。
8. 使用异常处理来捕获和处理运行时错误。将会抛出异常的代码放在 try 子句中，在 except 子句中罗列出异常，并且在 except 子句中处理异常。
9. Python 提供有像 ZeroDivisionError、SyntaxError 和 RuntimeError 这样的内置异常类。所有的 Python 异常类都直接或间接继承自 BaseException 类。也可以定义用户自己的异常类，自定义异常类扩展自 BaseException 类或它的子类，像 RuntimeError。
10. 可以使用 Python 的 pickle 模块将对象存入一个文件。dump 函数将对象写入文件而 load 函数将对象从文件中读出。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

第 13.2 ～ 13.5 节

**13.1 （文本删除）编写一个程序将某个指定字符串从一个文本文件中所有出现的地方删除。程序应该提示用户输入一个文件名和要删除的字符串。下面是一个运行实例。

```
Enter a filename: test.txt Enter
Enter the string to be removed: morning Enter
Done
```

*13.2 （统计一个文件中的字符数、单词数和行数）编写程序统计一个文件中的字符数、单词数以及行数。单词由空格分隔。程序应当提示用户输入一个文件名。下面是一个运行实例。

```
Enter a filename: test.txt Enter
1777 characters
210 words
71 lines
```

*13.3 （处理文本文件中的分数）假定一个文本文件中包含未指定个数的分数。编写一个程序，从文件读入分数，然后显示它们的和以及平均值。分数之间用空格分开。程序应当提示用户输入一个文件名。下面是一个运行实例。

```
Enter a filename: scores.txt Enter
There are 70 scores
The total is 800
The average is 33.33
```

*13.4 （写 / 读数据）编写一个程序，将随机产生的 100 个整数写入一个文件。文件中的整数由空格分开。从文件中读回数据，然后显示排好序的数据。程序应当要求提示用户输入一个文件名。如果文件已经存在，不能覆盖它。下面是一个运行实例。

```
Enter a filename: test.txt Enter
The file already exists
```

```
Enter a filename: test1.txt Enter
20 34 43 ... 50
```

**13.5 （替换文本）编写一个程序替换一个文件中的文本。程序应当提示用户输入一个文件名、一个旧的字符串和一个新字符串。下面是一个运行实例。

```
Enter a filename: test.txt Enter
Enter the old string to be replaced: morning Enter
Enter the new string to replace the old string: afternoon Enter
Done
```

*13.6 （统计单词数）编写一个程序来统计来自 http://cs.armstrong.edu/liang/data/Lincoln.txt 的美国总统亚伯拉罕林肯的葛底斯堡地址中单词的个数。

**13.7 （游戏：刽子手）改写编程题 10.29。程序读取存储在一个名为 hangman.txt 的文本文件中的单词。这些单词用空格分隔。

13.8 （文件加密）通过对文件中的每比特加 5 来给文件加密。编写一个程序提示用户输入一个输入文件名和一个输出文件名并且将加密后的输入文件版本存储在输出文件中。

13.9 （文件解密）假设一个文件已经使用编程题 13.8 中的方案进行了加密。编写一个程序来对这个加密文件解密。程序应当提示用户输出一个输入文件名和一个输出文件名并将解密后的输入文件版本存储在输出文件当中。

第 13.6 ～ 13.9 节

13.10 （Rational 类）修改程序清单 8-4 中的 Rational 类（Rational.py）来使程序在分母为 0 时抛出一个 RuntimeError 异常。

13.11 （Triangle 类）修改编程题 12.1 中的 Triangle 类来使程序在给定的三条边不能形成一个三角形时抛出一个 RuntimeError 异常。

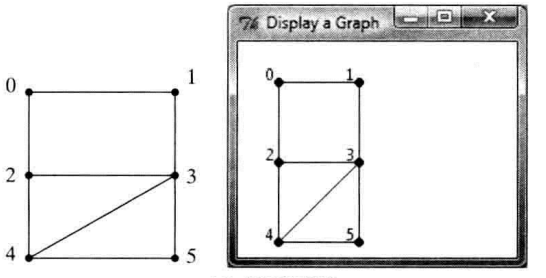
13.12 （TriangleError 类）定义一个名为 TriangleError 的异常类，该类扩展自 RuntimeError。TriangleError 类包含三角形三条边的私有数据域 side1、side2 和 side3 以及对它们的访问方法。修改编程题 12.1 中的 Triangle 类来在给定的三条边不能形成一个三角形时抛出一个 TriangleError 异常。

第 13.10 ～ 13.11 节

**13.13 （Tkinter：显示一个图形）一个图形包括一些端点以及连接这些点的线。编写程序从文件中读取一个图形，然后在面板上显示它。文件的第一行包括表明端点个数的数字（n）。这些端点都被标记上 0、1、…、n-1。每一个连接线的形式都是 u x y v1 ，v2，…，这种形式描述点 u 的位置在（x,y）处并且又和它相连的两条线是（u,v1）、（u,v2），依此类推。图 13-12a 给出一个图形对应文件的例子。程序提示用户输入文件名，从文件读取数据，然后在面板上显示这个图形，如图 13-12b。

```
File
6
0 30 30 1 2
1 90 30 0 3
2 30 90 0 3 4
3 90 90 1 2 4 5
4 30 150 2 3 5
5 90 150 3 4
```

a）图形对应文件



b）显示图形

图　13-12

**13.14 （Tkinter：显示一个图形）重写编程题 13.13 中的程序，从一个像 http://cs.armstrong.edu/liang/data/graph.txt 这样的网页 URL 来读取数据。程序提示用户输入文件所在的 URL 地址。

**13.15 （Tkinter：地址簿）重写第 13.11 节的地址簿实例研究，对它进行以下的改进，如图 13-13 所示。

a）添加一个新的名为“Update”的按钮。单击它可以让用户更新当前显示的地址。

b）在按钮下面添加一个标签来显示当前地址位置和列表中总的地址数。

c）实现程序清单 13-19 中未完成的 processPrevious 和 processLast 方法。

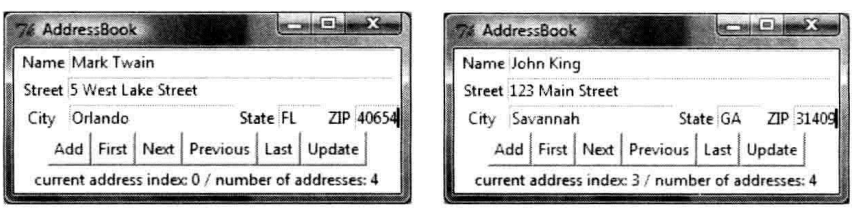


图 13-13　在地址簿 UI 上添加新的 Update 按钮和状态标签

**13.16 （创建大数据集）创建一个有 1000 行的数据文件。文件中的每一行都是由教师的姓、名、职称和工资组成。第 i 行教师的姓和名假设为 FirstNamei 和 LastNamei。职称随机生成为助教、副教授和教授。他们的工资也是随机生成的一个小数点后保留两位的数字。助教的工资应该在 50 000 到 80 000 之间，副教授的工资在 60 000 到 110 000 之间，而教授的工资在 75 000 到 130 000 之间。将文件存入 Salary.txt 中。下面是一些实例数据。

```
FirstName1 LastName1 assistant 60055.95
FirstName2 LastName2 associate 81112.45
...
FirstName1000 LastName1000 full 92255.21
```

*13.17 （处理大数据集）某大学将它的职工工资放在 http://cs.armstrong.edu/liang/data/Salary.txt 网站上。文件中的每一行都由教师的姓、名、职称和工资组成（参见编程题 13.16）。编写一个程序分别显示所有助教、所有副教授、所有教授以及所有教师的总工资，然后分别显示所有助教、所有副教授、所有教授以及所有教师他们各自的平均工资。

第三部分

Introduction to Programming Using Python

数据结构和算法

第 14 章
Introduction to Programming Using Python

元组、集合和字典

学习目标

- 创建元组（第 14.2 节）。
- 使用元组作为固定列表来防止添加、删除或替换元素（第 14.2 节）。
- 将常见的序列运算应用于元组（第 14.2 节）。
- 创建集合（第 14.3.1 节）。
- 使用 add 和 remove 方法对集合进行元素的添加和删除（第 14.3.2 节）。
- 使用 len、min、max 和 sum 函数对集合进行操作（第 14.3.2 节）。
- 使用 in 和 not in 运算符判断一个元素是否在一个集合中（第 14.3.2 节）。
- 使用 for 循环遍历集合中的元素（第 14.3.2 节）。
- 使用 issubset 或 issuperset 方法检测一个集合是否是另一个集合的子集或父集（第 14.3.3 节）。
- 使用 == 运算符检测两个集合是否具有相同的内容（第 14.3.4 节）。
- 使用运算符 |、&、- 和 ^ 实现集合求并、交、差和对称差（第 14.3.5 节）。
- 比较集合和列表的性能区别（第 14.4 节）。
- 使用集合开发一个统计 Python 源文件中关键字个数的程序（第 14.5 节）。
- 创建字典（第 14.6.1 节）。
- 使用语法 dictionaryName[key] 对字典添加、修改和获取元素（第 14.6.2 节）。
- 使用 del 关键字删除字典中的条目（第 14.6.3 节）。
- 使用 for 循环遍历字典中的关键字（第 14.6.4 节）。
- 使用 len 函数获取字典的大小（第 14.6.5 节）。
- 使用 in 或 not in 运算符检测一个关键字是否在字典中（第 14.6.6 节）。
- 使用 == 运算符检测两个字典是否有相同的内容（第 14.6.7 节）。
- 在字典上使用 keys、values、items、clean、get、pop 和 popitem 方法（第 14.6.8 节）。
- 使用字典开发应用程序（第 14.7 节）。

14.1 引言

关键点：可以使用元组存储一个固定的元素列表，使用集合存储和快速访问不重复的元素，使用字典存储键值对并使用这些关键字来快速访问元素。

“No-Fly”是一个列表，它是由美国政府恐怖分子筛查中心（Terrorist Screening Center）维护的不准登上商务飞机飞进和飞离美国的人员名单。假设我们需要编写一个程序来检测一个人是否在 No-Fly 列表上。可以使用一个 Python 列表来存储 No-Fly 列表上的名字。但是，对这个应用而言效率更高的数据结构是集合。在计算机科学中，*数据结构*（data structure）是用来在计算机上存储和组织数据的一种特殊方式，以便它可以有效地应用于特定的场合。

本章除了介绍集合，还介绍了两个其他有用的数据结构——元组和字典。

14.2　元组

关键点：元组跟列表类似，但是元组中的元素是固定的；也就是说，一旦一个元组被创建，就无法对元组中的元素进行添加、删除、替换或重新排序。

如果在应用中不应该对列表中的内容进行修改，那么就可以使用元组来防止元素被意外添加、删除或替换。除了元组的元素是固定的以外，元组与列表很像。进一步讲，由于 Python 的实现，元组比列表的效率更高。

可以通过将元素用一对括号括起来来创建一个元组。这些元素用逗号分隔。可以创建一个空元组或从一个列表创建一个元组，如下面的例子所示。

```
t1 = () # Create an empty tuple

t2 = (1, 3, 5) # Create a tuple with three elements

# Create a tuple from a list
t3 = tuple([2 * x for x in range(1, 5)])
```

也可以从一个字符串创建一个元组。字符串中的每个字就变成了元组的一个元素。例如：

```
# Create a tuple from a string
t4 = tuple("abac") # t4 is ['a', 'b', 'a', 'c']
```

元组是序列。在表 10-1 中针对序列的常见操作也可以用在元组上。可以在元组上使用 len、min、max 和 sum 函数。可以使用一个 for 循环遍历一个元组的所有元素，并使用一个下标运算符来访问元组中对应的元素或元素段。可以使用 in 和 not in 运算符来判断一个元素是否在元组中，并使用比较运算符来对元组中的元素进行比较。

程序清单 14-1 给出了一个使用元组的例子。

程序清单 14-1　TupleDemo.py

```
1  tuple1 = ("green", "red", "blue") # Create a tuple
2  print(tuple1)
3
4  tuple2 = tuple([7, 1, 2, 23, 4, 5]) # Create a tuple from a list
5  print(tuple2)
6
7  print("length is", len(tuple2)) # Use function len
8  print("max is", max(tuple2)) # Use max
9  print("min is", min(tuple2)) # Use min
10 print("sum is", sum(tuple2)) # Use sum
11
12 print("The first element is", tuple2[0]) # Use index operator
13
14 tuple3 = tuple1 + tuple2 # Combine two tuples
15 print(tuple3)
16
17 tuple3 = 2 * tuple1 # Duplicate a tuple
18 print(tuple3)
19
20 print(tuple2[2 : 4]) # Slicing operator
21 print(tuple1[-1])
22
23 print(2 in tuple2) # in operator
24
25 for v in tuple1:
```

```
26      print(v, end = ' ')
27  print()
28
29  list1 = list(tuple2) # Obtain a list from a tuple
30  list1.sort()
31  tuple4 = tuple(list1)
32  tuple5 = tuple(list1)
33  print(tuple4)
34  print(tuple4 == tuple5) # Compare two tuples
```

```
('green', 'red', 'blue')
(7, 1, 2, 23, 4, 5)
length is 6
max is 23
min is 1
sum is 42
The first element is 7
('green', 'red', 'blue', 7, 1, 2, 23, 4, 5)
('green', 'red', 'blue', 'green', 'red', 'blue')
(2, 23)
blue
True
green red blue
(1, 2, 4, 5, 7, 23)
True
```

程序利用一些字符串创建元组 tuple1（第 1 行），利用一个列表创建元组 tuple2（第 4 行）。程序对 tuple2 使用了 len、max、min 和 sum 函数（第 7 ～ 10 行）。可以使用下标运算符来访问一个元组中的元素（第 12 行），+ 运算符被用来合并两个元组（第 14 行），* 运算符被用来复制一个元组（第 17 行），而切片运算符用来获取元组的一部分（第 20 ～ 21 行）。可以使用 in 运算符来判断某个指定元素是否在一个元组中（第 23 行）。可以使用一个 for 循环来遍历一个元组中的元素（第 25 ～ 26 行）。

程序创建了一个列表（第 29 行），对这个列表进行分类（第 30 行），然后从这个列表创建两个元组（第 31 ～ 32 行）。使用比较操作符 == 对元组进行比较（第 34 行）。

元组的元素是固定的。那么第 17 行的语句不会因为 tuple3 已经在第 14 行定义而抛出一个错误吗？但是，第 17 行的语句没有错误，因为重新分配了一个新元组给变量 tuple3。现在，tuple3 指向新元组。“元组的元素是固定的”是指不能给一个元组添加、删除和替换元素以及打乱元组中的元素。

注意：一个元组包含了一个固定的元素列表。一个元组里的一个个体元素可能是易变的。例如，下面的代码创建了一个圆的元组（第 2 行），并改变第一个圆的半径为 30（第 3 行）。

```
1  >>> from CircleFromGeometricObject import Circle
2  >>> circles = (Circle(2), Circle(4), Circle(7))
3  >>> circles[0].setRadius(30)
4  >>> circles[0].getRadius()
5  >>> 30
6  >>>
```

在这个例子中，元组中每一个元素都是一个圆对象。尽管不能添加、删除或替换元组中的圆对象，但是可以改变一个圆的半径，因为一个圆对象是可变的。如果一个元组包含不可变的对象，那么这个元组被称为不可变的。例如，一个数字元组或一个字符串元组是不可变的。

检查点

14.1　列表和元组的区别是什么？如何从列表创建元组？如何从元组创建列表？

14.2　下面代码的错误是什么？

```
t = (1, 2, 3)
t.append(4)
t.remove(0)
t[0] = 1
```

14.3　下面的代码正确吗？

```
t1 = (1, 2, 3, 7, 9, 0, 5)
t2 = (1, 2, 5)
t1 = t2
```

14.4　给出下面代码的输出。

```
t = (1, 2, 3, 7, 9, 0, 5)
print(t)
print(t[0])
print(t[1: 3])
print(t[-1])
print(t[ : -1])
print(t[1 : -1])
```

14.5　给出下面代码的输出？

```
t = (1, 2, 3, 7, 9, 0, 5)
print(max(t))
print(min(t))
print(sum(t))
print(len(t))
```

14.6　给出下面代码的输出？

```
t1 = (1, 2, 3, 7, 9, 0, 5)
t2 = (1, 3, 22, 7, 9, 0, 5)
print(t1 == t2)
print(t1 != t2)
print(t1 > t2)
print(t1 < t2)
```

14.3　集合

关键点：集合与列表类似，可以使用它们存储一个元素集合。但是，不同于列表，集合中的元素是不重复且不是按任何特定顺序放置的。

如果你的应用程序不关心元素的顺序，使用一个集合来存储元素比使用列表效率更高。本节介绍如何使用集合。

14.3.1　创建集合

可以通过将元素用一对花括号（{}）括起来以创建一个元素集合。集合中的元素用逗号分隔。可以创建一个空集，或者从一个列表或一个元组创建一个集合，如下面的例子所示。

```
s1 = set() # Create an empty set
s2 = {1, 3, 5} # Create a set with three elements
s3 = set((1, 3, 5)) # Create a set from a tuple

# Create a set from a list
s4 = set([x * 2 for x in range(1, 10)])
```

同样的，可以通过使用语法 list(set) 或 tuple(set) 从集合创建一个列表或一个元组。

也可以从一个字符串创建一个集合。字符串中的每个字符就成为集合中的一个元素。例如：

```
# Create a set from a string
s5 = set("abac") # s5 is {'a', 'b', 'c'}
```

注意：尽管字符 a 在字符串中出现了两次，但在集合中只出现了一次，因为一个集合中不存储重复的元素。

一个集合可以包含类型相同或不同的元素。例如：s = {1,2,3,"one","two","three"} 是一个包含数字和字符串的集合。集合中的每个元素必须是哈希的（hashable）。Python 中的每一个对象都有一个哈希值，而且如果在对象的生命周期里对象的哈希值从未改变，那么这个对象是哈希的。目前所介绍的所有类型对象除了列表之外都是哈希的。为什么集合元素必须是哈希的这个问题将在本书对应网站的第 21 章“哈希：实现集合和字典”中解释。

14.3.2　操作和访问集合

可以通过使用 add(e) 或 remove(e) 方法来对一个集合添加或删除元素。可以使用函数 len、min、max 和 sum 对集合操作，可以使用 for 循环遍历一个集合中的所有元素。

可以使用 in 或 not in 运算符来判断一个元素是否在一个集合当中。例如：

```
>>> s1 = {1, 2, 4}
>>> s1.add(6)
>>> s1
{1, 2, 4, 6}
>>> len(s1)
4
>>> max(s1)
6
>>> min(s1)
1
>>> sum(s1)
13
>>> 3 in s1
False
>>> s1.remove(4)
>>> s1
{1, 2, 6}
>>>
```

注意：如果删除一个集合中不存在的元素，remove(e) 方法将抛出一个 KeyError 异常。

14.3.3　子集和超集

如果集合 s1 中的每个元素都在集合 s2 中，则称 s1 是 s2 的子集。可以使用 s1.issubset(s2) 方法来判断 s1 是否是 s2 的子集，如下面代码所示。

```
>>> s1 = {1, 2, 4}
>>> s2 = {1, 4, 5, 2, 6}
>>> s1.issubset(s2) # s1 is a subset of s2
True
>>>
```

如果一个集合 s2 中的元素同样都在集合 s1 中，则称集合 s1 是集合 s2 的超集。可以使用 s1.issuperset(s2) 方法来判断 s1 是否是 s2 的超集，如下面代码所示。

```
>>> s1 = {1, 2, 4}
>>> s2 = {1, 4, 5, 2, 6}
>>> s2.issuperset(s1) # s2 is a superset of s1
True
>>>
```

14.3.4　相等性测试

可以使用运算符 == 和 != 来检测两个集合是否包含相同的元素。例如：

```
>>> s1 = {1, 2, 4}
>>> s2 = {1, 4, 2}
>>> s1 == s2
True
>>> s1 != s2
False
>>>
```

在这个例子中，尽管 s1 和 s2 的元素顺序不同，但是这两个集合包含相同的元素。

注意：使用传统的比较运算符（>、>=、<= 和 <）来比较集合毫无意义，因为集合中的元素并没有排序。但是，当这些操作符用在集合上时有着特殊的含义：

- 如果 s1 是 s2 的一个真子集，则 s1<s2 返回 True。
- 如果 s1 是 s2 的一个子集，则 s1<=s2 返回 True。
- 如果 s1 是 s2 的一个真超集，则 s1>s2 返回 True。
- 如果 s1 是 s2 的一个超集，则 s1>=s2 返回 True。

注意：如果 s1 是 s2 的一个真子集，那么 s1 的每个元素同样也都在 s2 中，但是 s2 中至少存在一个不在 s1 中的元素。如果 s1 是 s2 的一个真子集，那么 s2 是 s1 的一个真超集。

14.3.5　集合运算

Python 提供了求并集、交集、差集和对称差集合的运算方法。

两个集合的并集是一个包含这两个集合所有元素的集合。可以使用 union 方法或者 | 运算符来实现这个操作。例如：

```
>>> s1 = {1, 2, 4}
>>> s2 = {1, 3, 5}
>>> s1.union(s2)
{1, 2, 3, 4, 5}
>>>
>>> s1 | s2
{1, 2, 3, 4, 5}
>>>
```

两个集合的交集是一个包含了两个集合共同的元素的集合。可以使用 intersection 方法或者 & 运算符来实现这个操作。例如：

```
>>> s1 = {1, 2, 4}
>>> s2 = {1, 3, 5}
>>> s1.intersection(s2)
{1}
>>>
>>> s1 & s2
{1}
>>>
```

set1 和 set2 之间的差集是一个包含了出现在 set1 但不出现在 set2 的元素的集合。可以使用 difference 方法或 - 运算符来实现这个操作。例如：

```
>>> s1 = {1, 2, 4}
>>> s2 = {1, 3, 5}
>>> s1.difference(s2)
{2, 4}
>>>
>>> s1 - s2
{2, 4}
>>>
```

两个集合之间的对称差（或者称为异或）集合是一个包含了除它们共同元素之外所有在这两个集合之中的元素。可以使用 symmertric_difference 方法或 ^ 运算符来实现这个操作。例如：

```
>>> s1 = {1, 2, 4}
>>> s2 = {1, 3, 5}
>>> s1.symmetric_difference(s2)
{2, 3, 4, 5}
>>>
>>> s1 ^ s2
{2, 3, 4, 5}
>>>
```

注意到这些 set 方法都返回一个结果集合，但是它们并不会改变这些集合中的元素。

程序清单 14-2 列举了使用集合的程序。

程序清单 14-2 SetDemo.py

```
1  set1 = {"green", "red", "blue", "red"} # Create a set
2  print(set1)
3
4  set2 = set([7, 1, 2, 23, 2, 4, 5]) # Create a set from a list
5  print(set2)
6
7  print("Is red in set1?", "red" in set1)
8
9  print("length is", len(set2)) # Use function len
10 print("max is", max(set2)) # Use max
11 print("min is", min(set2)) # Use min
12 print("sum is", sum(set2)) # Use sum
13
14 set3 = set1 | {"green", "yellow"} # Set union
15 print(set3)
16
17 set3 = set1 - {"green", "yellow"} # Set difference
18 print(set3)
19
20 set3 = set1 & {"green", "yellow"} # Set intersection
21 print(set3)
22
23 set3 = set1 ^ {"green", "yellow"} # Set exclusive or
24 print(set3)
25
26 list1 = list(set2) # Obtain a list from a set
27 print(set1 == {"green", "red", "blue"}) # Compare two sets
28
29 set1.add("yellow")
30 print(set1)
31
```

```
32  set1.remove("yellow")
33  print(set1)
```

```
{'blue', 'green', 'red'}
{1, 2, 4, 5, 7, 23}
Is red in set1? True
length is 6
max is 23
min is 1
sum is 42
{'blue', 'green', 'yellow', 'red'}
{'blue', 'red'}
{'green'}
{'blue', 'red', 'yellow'}
True
{'blue', 'green', 'yellow', 'red'}
{'blue', 'green', 'red'}
```

这个程序创建集合 set1 为 {"green","red","blue","red"}（第 1 行）。因为一个集合中的元素都不重复，因此只有一个 red 元素被存储在集合 set1 中。这个程序使用 set 函数从一个列表创建集合 set2（第 4 行）。

这个程序对集合应用函数 len、max、min 和 sum（第 9 ～ 12 行）。注意：不能使用下标运算符来访问一个集合中的元素，因为这些元素并没有特定的顺序。

程序在第 14 ～ 24 行实现求并集、差集、交集和对称差集合的操作。

```
Set union:  {"green", "red", "blue"} | {"green", "yellow"})
            => {"green", "red", "blue", "yellow"} (line 14)

Set difference:  {"green", "red", "blue"} - {"green", "yellow"})
                 => {"red", "blue"} (line 17)

Set intersection:  {"green", "red", "blue"} & {"green", "yellow"})
                   => {"green"} (line 20)

Set symmetric_difference:  {"green", "red", "blue"} ^ {"green", "yellow"})
                           => {"red", "blue", "yellow"} (line 23)
```

程序使用 == 来判断两个集合是否具有相同的元素（第 27 行）。

使用 add 和 remove 方法来对一个集合进行元素进行添加和删除（第 29、32 行）。

检查点

14.7　如何创建一个空集合？

14.8　列表、集合或元组能有不同类型的元素吗？

14.9　下面哪个集合是被正确创建的？

```
s = {1, 3, 4}
s = {{1, 2}, {4, 5}}
s = {[1, 2], [4, 5]}
s = {(1, 2), (4, 5)}
```

14.10　列表和集合的区别是什么？如何从列表创建集合？如何从集合创建列表？如何从集合创建元组？

14.11　给出下面代码的输出。

```
students = {"peter", "john"}
print(students)
students.add("john")
print(students)
students.add("peterson")
```

```
print(students)
students.remove("peter")
print(students)
```

14.12　下面的代码会出现运行时错误吗？

```
students = {"peter", "john"}
students.remove("johnson")
print(students)
```

14.13　给出下面代码的输出。

```
student1 = {"peter", "john", "tim"}
student2 = {"peter", "johnson", "tim"}
print(student1.issuperset({"john"}))
print(student1.issubset(student2))
print({1, 2, 3} > {1, 2, 4})
print({1, 2, 3} < {1, 2, 4})
print({1, 2} < {1, 2, 4})
print({1, 2} <= {1, 2, 4})
```

14.14　给出下面代码的输出。

```
numbers = {1, 4, 5, 6}
print(len(numbers))
print(max(numbers))
print(min(numbers))
print(sum(numbers))
```

14.15　给出下面代码的输出。

```
s1 = {1, 4, 5, 6}
s2 = {1, 3, 6, 7}
print(s1.union(s2))
print(s1 | s2)
print(s1.intersection(s2))
print(s1 & s2)
print(s1.difference(s2))
print(s1 - s2)
print(s1.symmetric_difference(s2))
print(s1 ^ s2)
```

14.16　给出下面代码的输出。

```
set1 = {2, 3, 7, 11}
print(4 in set1)
print(3 in set1)
print(len(set1))
print(max(set1))
print(min(set1))
print(sum(set1))
print(set1.issubset({2, 3, 6, 7, 11}))
print(set1.issuperset({2,3, 7, 11}))
```

14.17　给出下面代码的输出。

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}

set3 = set1 | set2

print(set1, set2, set3)

set3 = set1 - set2
print(set1, set2, set3)
```

```
set3 = set1 & set2
print(set1, set2, set3)

set3 = set1 ^ set2
print(set1, set2, set3)
```

14.4　比较集合和列表的性能

关键点：对于 in 和 not in 运算符和 remove 方法，集合比列表的效率更高。

列表中的元素可以使用下标运算符来访问。但是，集合并不支持下标运算符，因为集合中的元素是无序的。使用 for 循环遍历集合中的所有元素。现在，我们进行一个有趣的实验来测试集合和列表的性能。程序清单 14-3 中的程序给出①检测一个元素是否在集合和列表中的各自的执行时间；②从集合和列表删除元素时各自的执行时间。

程序清单 14-3　SetListPerformanceTest.py

```
import random
import time

NUMBER_OF_ELEMENTS = 10000

# Create a list
lst = list(range(NUMBER_OF_ELEMENTS))
random.shuffle(lst)

# Create a set from the list
s = set(lst)

# Test if an element is in the set
startTime = time.time() # Get start time
for i in range(NUMBER_OF_ELEMENTS):
    i in s
endTime = time.time() # Get end time
runTime = int((endTime - startTime) * 1000) # Get test time
print("To test if", NUMBER_OF_ELEMENTS,
    "elements are in the set\n",
    "The runtime is", runTime, "milliseconds")

# Test if an element is in the list
startTime = time.time() # Get start time
for i in range(NUMBER_OF_ELEMENTS):
    i in lst
endTime = time.time() # Get end time
runTime = int((endTime - startTime) * 1000) # Get test time
print("\nTo test if", NUMBER_OF_ELEMENTS,
    "elements are in the list\n",
    "The runtime is", runTime, "milliseconds")

# Remove elements from a set one at a time
startTime = time.time() # Get start time
for i in range(NUMBER_OF_ELEMENTS):
    s.remove(i)
endTime = time.time() # Get end time
runTime = int((endTime - startTime) * 1000) # Get test time
print("\nTo remove", NUMBER_OF_ELEMENTS,
    "elements from the set\n",
    "The runtime is", runTime, "milliseconds")

# Remove elements from a list one at a time
startTime = time.time() # Get start time
```

```
for i in range(NUMBER_OF_ELEMENTS):
    lst.remove(i)
endTime = time.time() # Get end time
runTime = int((endTime - startTime) * 1000) # Get test time
print("\nTo remove", NUMBER_OF_ELEMENTS,
    "elements from the list\n",
    "The runtime is", runTime, "milliseconds")
```

```
To test if 10000 elements are in the set
The runtime is 5 milliseconds

To test if 10000 elements are in the list
The runtime is 4274 milliseconds

To remove 10000 elements from the set
The runtime is 7 milliseconds

To remove 10000 elements from the list
The runtime is 1853 milliseconds
```

在第 7 行，range(NUMBER_OF_ELEMENTS) 函数返回从 0 到 NUMBER_OF_ELEMENTS−1 的数字序列。因此，list(range(NUMBER_OF_ELEMENTS)) 返回一个从 0 到 NUMBER_OF_ELEMENTS−1 的整数列表（第 7 行）。程序将这个列表打乱（第 8 行），并通过这个列表创建一个集合（第 11 行）。现在，这个集合和列表都包含有相同的元素。

程序获取测试元素 0 到 NUMBER_OF_ELEMENTS−1 是否在集合中（第 14 ～ 21 行）和是否在列表中（第 24 ～ 31 行）的执行时间。如在输出中所看到的，集合对应的执行时间是 5 毫秒而列表对应的执行时间是 4274 毫秒。

程序获取删除集合和列表中的元素 0 ～ NUMBER_OF_ELEMENTS−1 各自的运行时间。同样的，如输出所示，集合对应的执行时间是 7 毫秒而列表对应的执行时间是 1853 毫秒。

正如上面的运行时间所示，检测一个元素是在一个集合中还是在一个列表中时，集合的效率比列表更高些。因此，本章开头提到的“No-Fly”列表应当使用一个集合而非一个列表来实现，因为检测元素是否存在时集合的速度更快。

你可能会想知道集合为什么比列表的效率高。为了得到这个答案，阅读配套网站上关于开发高效算法、链表和哈希的章节。

14.5　实例研究：统计关键字

关键点：本节列出了一个统计 Python 源文件中关键字个数的应用程序。

对于 Python 源文件中的每个单词，需要判断这个单词是否是一个关键字。为了有效处理这个问题，将所有关键字存入一个集合当中并使用 in 运算符来检测一个单词是否在这个关键字集合中。程序如程序清单 14-4 所示。

程序清单 14-4　CountKeywords.py

```
import os.path
import sys

def main():
    keyWords = {"and", "as", "assert", "break", "class",
                "continue", "def", "del", "elif", "else",
                "except", "False", "finally", "for", "from",
                "global", "if", "import", "in", "is", "lambda",
                "None", "nonlocal", "not", "or", "pass", "raise",
                "return", "True", "try", "while", "with", "yield"}

```

```
12      filename = input("Enter a Python source code filename: ").strip()
13
14      if not os.path.isfile(filename): # Check if file exists
15          print("File", filename, "does not exist")
16          sys.exit()
17
18      infile = open(filename, "r") # Open files for input
19
20      text = infile.read().split() # Read and split words from the file
21
22      count = 0
23      for word in text:
24          if word in keyWords:
25              count += 1
26
27      print("The number of keywords in", filename, "is", count)
28
29  main()
```

```
Enter a Python source code filename: GuessNumber.py Enter
The number of keywords in GuessNumber.py is 7
```

```
Enter a Python source file: TTT.py Enter
File TTT.py does not exist
```

程序创建了一个关键字集合（第 5 ～ 10 行）并提示用户输入一个 Python 源文件名（第 12 行）。它检测这个文件是否存在（第 14 行）。如果不存在，退出这个程序（第 16 行）。

程序打开这个文件并从这个文本中提取出所有单词（第 20 行）。对于每一个单词，程序将检测这个单词是否是一个关键字（第 24 行）。如果是，将计数器加 1（第 25 行）。

14.6　字典

关键点：一个字典是一个存储键值对集合的容器对象。它通过使用关键字实现快速获取、删除和更新值。

假设程序需要存储“No-Fly”表中有关恐怖分子的详细信息。字典就是这个任务的一种有效的数据结构。一个字典是按照关键字存储值的集合。这些关键字很像下标运算符。在一个列表中，下标是整数。在一个字典中，关键字必须是一个可哈希对象。一个字典不能包含有重复的关键字。每个关键字都对应着一个值。一个关键字和它对应的值形成存储在字典中的一个条目（输入域），如图 14-1a 所示。这种数据结构被称为“字典”，因为它与词典很类似，在这里，单词就相当于关键字而这些单词的详细定义就是相应的值。一个字典也被认为是一张图，它将每个关键字和一个值相匹配。

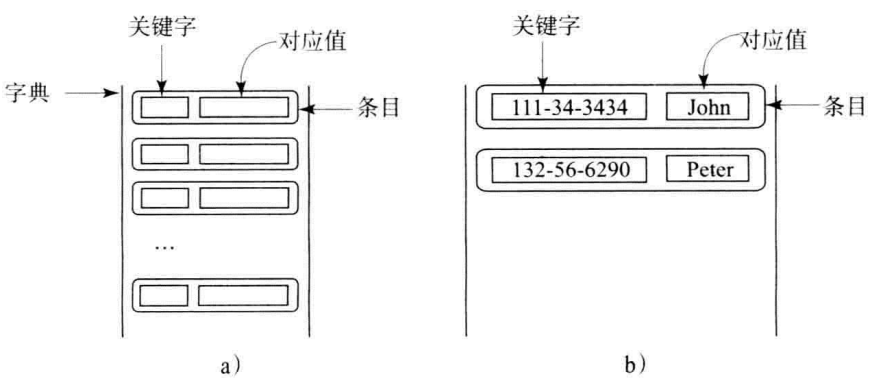


图 14-1　字典中的一个条目是一个键值对

14.6.1　创建一个字典

可以通过一对花括号（{}）将这些条目括起来以创建一个字典。每一个条目都由一个关键字，然后跟着一个冒号，再跟着一个值组成。每一个条目都用逗号分隔。例如，下面语句：

```
students = {"111-34-3434":"John", "132-56-6290":"Peter"}
```

创建一个具有两个条目的字典，如图 14-1b 所示。字典中的每一个条目的形式都是 key：value。第一个条目的关键字是 111-34-3434，它对应的值是 John。关键字必须是可哈希类型，例如：数字和字符串。而值可以是任意类型。

可以使用下面的语法来创建一个空字典。

```
students = {} # Create an empty dictionary
```

注意：Python 使用花括号创建集合和字典。语法 {} 被用来表示一个空字典。为了创建一个空集合，使用 set()。

14.6.2　添加、修改和获取值

为了添加一个条目到字典中，使用语法：

```
dictionaryName[key] = value
```

例如：

```
students["234-56-9010"] = "Susan"
```

如果这个关键字已经在字典中存在，前面的语法将替换该关键字对应的值。

为了获取一个值，只要使用 dictionaryName[key] 编写一个表达式即可。如果该关键字在字典中，那么返回这个关键字对应的值。否则，抛出一个 KeyError 异常。

例如：

```
 1  >>> students = {"111-34-3434":"John", "132-56-6290":"Peter"}
 2  >>> students["234-56-9010"] = "Susan" # Add a new item
 3  >>> students["234-56-9010"]
 4  "Susan"
 5  >>> students["111-34-3434"] = "John Smith"
 6  >>> students["111-34-3434"]
 7  "John Smith"
 8  >>> student["343-45-5455"]
 9  Traceback (most recent call last):
10    File "<stdin>", line 1, in <module>
11  KeyError: '343-45-5455'
12  >>>
```

第 1 行创建带两个条目的字典。第 2 行添加一个关键字为 234-56-9010 且值为 Susan 的条目。第 3 行返回和关键字 234-56-9010 相关的值。第 5 行使用新的数据值 John Smith 修改关键字 111-34-3434 对应的值，而第 8 行获取一个不存在关键字 343-45-5455 对应的值，这会抛出一个 KeyError 异常。

14.6.3　删除条目

为了从字典删除一个条目，使用语法：

```
del dictionaryName[key]
```

例如：

```
del students["234-56-9010"]
```

这条语句从字典中删除关键字为 234-56-9010 的对应条目。如果字典中不存在该关键字，那么抛出一个 KeyError 异常。

14.6.4　循环条目

可以使用一个 for 循环来遍历字典中所有的关键字。例如：

```
1  >>> students = {"111-34-3434":"John", "132-56-6290":"Peter"}
2  >>> for key in students:
3  ...     print(key + ":" + str(students[key]))
4  ...
5  "111-34-3434":"John"
6  "132-56-6290":"Peter"
7  >>>
```

for 循环对字典 students 中的关键字进行迭代（第 2 行）。students[key] 返回关键字 key 对应的值（第 3 行）。

14.6.5　len 函数

可以使用 len(dictionary) 来获得一个字典中条目的数目。例如：

```
1  >>> students = {"111-34-3434":"John", "132-56-6290":"Peter"}
2  >>> len(students)
3  2
4  >>>
```

在第 2 行，len(students) 返回字典 students 中条目的数目。

14.6.6　检测一个关键字是否在字典中

可以使用 in 或 not in 运算符来判断一个关键字是否在一个字典当中。例如：

```
1  >>> students = {"111-34-3434":"John", "132-56-6290":"Peter"}
2  >>> "111-34-3434" in students
3  True
4  >>> "999-34-3434" in students
5  False
6  >>>
```

在第 2 行，"111-34-3434" in students 将检测关键字 111-34-3434 是否在字典 students 中。

14.6.7　相等性检测

可以使用运算符 == 和 != 来检测两个字典是否包含同样的条目。例如：

```
>>> d1 = {"red":41, "blue":3}
>>> d2 = {"blue":3, "red":41}
>>> d1 == d2
True
>>> d1 != d2
False
>>>
```

在这个例子中，不管这些条目在字典中的顺序，d1 和 d2 包含有相同的条目。

注意：不能使用比较运算符（>、>=、<=和<）对字典进行比较，因为字典中的条目是没有顺序的。

14.6.8　字典方法

Python 中的字典类是 dict。图 14-2 罗列出能够被一个字典对象调用的方法。

dict	
keys(): tuple	返回一个关键字序列
values(): tuple	返回一个值序列
items(): tuple	返回一个元组序列，每个元组都是一个条目的（键，值）
clear(): None	删除所有条目
get(key): value	返回这个关键字对应的值
pop(key): value	删除这个关键字对应的条目并返回它的值
popitem(): tuple	返回一个随机选择的键值对作为元组并删除这个被选择的条目

图 14-2　dict 类提供了操作一个字典对象的方法

get(key) 方法除了当关键字 key 不在字典中时返回 None 而不是抛出一个异常，其他都与 dictionaryName[key] 类似。pop(key) 方法与 del dictionaryName[key] 类似。

下面是使用这些方法的一些例子。

```
1  >>> students = {"111-34-3434":"John", "132-56-6290":"Peter"}
2  >>> tuple(students.keys())
3  ("111-34-3434", "132-56-6290")
4  >>> tuple(students.values())
5  ("John", "Peter")
6  >>> tuple(students.items())
7  (("111-34-3434", "John"), ("132-56-6290", "Peter"))
8  >>> students.get("111-34-3434")
9  "John"
10 >>> print(students.get("999-34-3434"))
11 None
12 >>> students.pop("111-34-3434")
13 "John"
14 >>> students
15 {"132-56-6290":"Peter"}
16 >>> students.clear()
17 >>> students
18 {}
19 >>>
```

字典 students 在第 1 行被创建，而第 2 行的 students.keys() 将返回字典中的关键值。在第 4 行，students.values() 返回字典中的值，而第 6 行的 students.items() 将字典中的条目作为元组返回。在第 10 行，调用 students.get（"999-34-3434"）返回关键字 999-34-3434 对应的学生姓名。第 12 行调用 students.pop（"111-34-3434"）来删除关键字 111-34-3434 对应的字典中的条目。在第 16 行，调用 studens.clear() 来删除字典中的所有条目。

检查点

14.18　如何创建一个空字典？

14.19　下面哪个字典是被正确创建的？

```
d = {1:[1, 2], 3:[3, 4]}
d = {[1, 2]:1, [3, 4]:3}
```

```
d = {(1, 2):1, (3, 4):3}
d = {1:"john", 3:"peter"}
d = {"john":1, "peter":3}
```

14.20　字典中每个条目都有两部分。它们被称作是什么？

14.21　假设一个名为 students 的字典是 {"john":3,"peter":2}。下面的语句实现什么功能？

(a) `students["susan"] = 5`

(b) `students["peter"] = 5`

(c) `students["peter"] += 5`

(d) `del students["peter"]`

14.22　假设一个名为 students 的字典是 {"john":3,"peter":2}。下面的语句实现什么功能？

(a) `print(len(students))`

(b) `print(students.keys())`

(c) `print(students.values())`

(d) `print(students.items())`

14.23　给出下面代码的输出。

```
def main():
    d = {"red":4, "blue":1, "green":14, "yellow":2}
    print(d["red"])
    print(list(d.keys()))
    print(list(d.values()))
    print("blue" in d)
    print("purple" in d)
    d["blue"] += 10
    print(d["blue"])

main() # Call the main function
```

14.24　给出下面代码的输出。

```
def main():
    d = {}
    d["susan"] = 50
    d["jim"] = 45
    d["joan"] = 54
    d["susan"] = 51
    d["john"] = 53
    print(len(d))

main() # Call the main function
```

14.25　对于一个字典 d，你可以使用 d[key] 或 d.get(key) 来返回这个关键字对应的值。它们之间的区别是什么？

14.7　实例研究：单词的出现次数

关键点：编写程序统计一个文本文件中单词的出现次数，并将出现次数最多的单词和它们的出现次数按降序显示。

这个实例研究中的程序使用一个字典来存储包含了单词和它的次数的条目。程序判断文件中的每个单词是否已经是字典中的一个关键字。如果不是，程序将添加一个条目，将这个单词作为该条目的关键字，并将它对应的值设置为 1。否则，程序将该单词（关键字）对应的值加 1。假设这些单词是不考虑大小写的（例如：认为 Good 与 good 是一样的）。这个程

序将出现次数最多的单词和它们的出现次数按降序显示。

程序清单 14-5 给出了这个问题的解决方法。

程序清单 14-5 CountOccurrenceOfWords.py

```
1  def main():
2      # Prompt the user to enter a file
3      filename = input("Enter a filename: ").strip()
4      infile = open(filename, "r") # Open the file
5
6      wordCounts = {} # Create an empty dictionary to count words
7      for line in infile:
8          processLine(line.lower(), wordCounts)
9
10     pairs = list(wordCounts.items()) # Get pairs from the dictionary
11
12     items = [[x, y] for (y, x) in pairs] # Reverse pairs in the list
13
14     items.sort() # Sort pairs in items
15
16     for i in range(len(items) - 1, len(items) - 11, -1):
17         print(items[i][1] + "\t" + str(items[i][0]))
18
19 # Count each word in the line
20 def processLine(line, wordCounts):
21     line = replacePunctuations(line) # Replace punctuation with space
22     words = line.split() # Get words from each line
23     for word in words:
24         if word in wordCounts:
25             wordCounts[word] += 1
26         else:
27             wordCounts[word] = 1
28
29 # Replace punctuation in the line with a space
30 def replacePunctuations(line):
31     for ch in line:
32         if ch in "~@#$%^&*()_-+=~<>?/,.;:!{}[]|'\"":
33             line = line.replace(ch, " ")
34
35     return line
36
37 main() # Call the main function
```

```
Enter a filename: Lincoln.txt ↵Enter
that    13
the     11
we      10
to      8
here    8
a       7
and     6
of      5
nation  5
it      5
```

程序提示用户输入一个文件名（第 3 行）并打开这个文件（第 4 行）。创建一个字典 wordCounts（第 6 行）来存储由单词和它们的出现次数构成的对。单词是字典的关键字。

程序从文件读取每一行并调用 processLine(line,wordCounts) 来统计这一行每个单词的出现次数（第 7 ～ 8 行）。假设字典 wordCounts 是 {"red":7,"blue":5,"green":2}。那么如何对它排序？字典对象没有 sort 方法。但是，list 对象有，因此，可以将字典的每一个对放入一

个列表中，然后对这个列表排序。这个程序在第 10 行获取到这个列表。如果使用 sort 方法对这个列表排序，程序将按每对的第一个元素进行排序，但是我们需要对出现次数进行排序（每对的第二个元素）。我们该怎么办呢？秘诀是可以将每对倒置。程序利用倒置每一对来创建一个新列表（第 12 行），然后应用 sort 方法（第 14 行）。现在，将对列表进行如下排序：[[2,"green"],[5,"blue"],[7,"red"]]。

这个程序显示出现次数最多的 10 组单词和它们的统计次数（第 16 ～ 17 行）。

processLine(line,wordCounts) 函数调用 replacePunctuations(line) 将所有的标点符号用空格替换（第 21 行），然后使用 split 方法提取单词（第 22 行）。如果一个单词已经存在字典中，那么程序将对它的计数器加 1(第 25 行)；否则，程序将给字典增加一个条目（第 27 行）。

replacePunctuations(line) 方法检测每一行的每个字符。如果它是一个标点符号，程序将用空格替换（第 32 ～ 33 行）。

现在坐下来思考一下，如果不使用字典该如何编写这个程序。可以使用像 [[key1,value1],[key2,value2],…] 这样的一个嵌套列表，但是新程序将会更长且更复杂。你将会发现，在解决类似这种问题时，字典是一种非常高效且功能强大的数据结构。

关键术语

data structure（数据结构）
dictionary（字典）
dictionary entry（字典输入域）
dictionary item（字典条目）
hashable（可哈希的）
immutable tuple（不可变元组）
key/value pair（键值对）
map（图）
set（集合）
set difference（集合的差）
set intersection（集合的交）
set union（集合的并）
set symmetric difference（集合的对称差）
tuple（元组）

本章总结

1. 一个元组是一个固定列表。不能对元组中的元素进行添加、删除或替换。
2. 由于元组是一个序列，所以序列的常用操作也可以用于元组。
3. 尽管不能对元组进行元素的添加、删除或者替换，但是如果该元素是可变的话你可以改变这个单独元素的内容。
4. 如果元组的所有元素都是不可变的，那么这个元组是不可变的。
5. 集合就像是用来存储元素集的列表。但是，不同于列表，集合中的元素是不可重复的而且是没有以特定顺序放置的。
6. 可以使用 add 方法向一个集合添加元素，使用 remove 方法从一个集合删除元素。
7. 函数 len、min、max 和 sum 都可用在集合上。
8. 可以使用一个 for 循环来遍历集合中的元素。
9. 可以使用 issubset 或 issuperset 方法来检测一个集合是否是另一个集合的子集或父集，并使用 |、&、- 和 ^ 运算符来实现求集合的并集、交集、差集和对称差集。
10. 在判断一个元素是否存在于集合或列表中，以及从集合或列表删除元素时，集合都比列表的效率更高。
11. 字典可用于存储键值对。可以使用一个关键字来获取一个值。这些关键字就像是一个下标操作符。在一个列表中，这些下标都是整数。在一个字典中，这些关键字可以是任意的可哈希对象，例如：数字和字符串。

12. 可以使用 dictionaryName[key] 来获取字典中某个给定关键字对应的值，并使用 dictionaryName[key]=value 来添加或修改字典中的一个条目。
13. 可以使用 del dictionaryName[key] 删除给定关键字对应的条目。
14. 可以使用一个 for 循环来遍历一个字典中的所有关键字。
15. 可以使用 len 函数返回字典的所有条目数。
16. 可以使用 in 和 not in 运算符来确定一个关键字是否在字典当中，使用 == 和 != 操作符来检测两个字典是否相同。
17. 可以对字典使用 keys()、values()、items()、clear()、get(key)、pop(key) 和 popitem() 方法。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

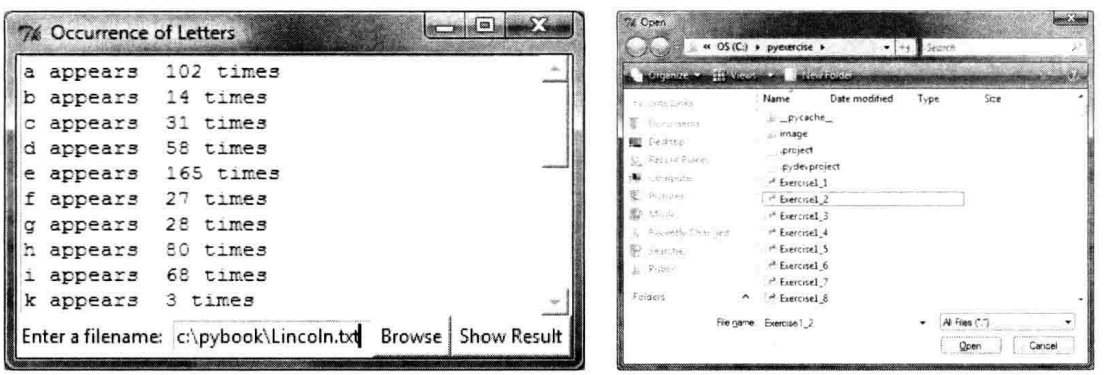
第 14.2 ～ 14.6 节

*14.1 （显示关键字）修改程序清单 14-4，显示一个 Python 源文件中的关键字以及这些关键字的出现次数。

*14.2 （数字的出现次数）编写一个程序读取未指定个数的整数，并找出出现次数最多的整数。例如：如果输入 2 3 40 3 5 4 -3 3 3 2 0，那么数字 3 出现的次数最多。在同一行里输入所有数字。如果不止一个数字的出现次数最多，那么这些数字都得显示。例如：数字 9 和 3 在列表 9 30 3 9 3 2 4 都出现了 2 次，那么应当把它们都显示出来。

*14.3 （统计每个关键字的出现次数）编写一个程序读入一个 Python 源代码文件并统计文件中每个关键字的出现次数。程序应当提示用户输入 Python 源代码文件名。

*14.4 （Tkinter：统计每个字母的出现次数）重写程序清单 14-5 中的程序，使用一个 GUI 程序让用户从一个输入域中输入文件名，如图 14-3a 所示。也可以通过单击“Browse”按钮显示一个打开文件对话框来选择一个文件，如图 14-3b 所示。然后所选择的文件显示在输入域上。单击“Show Result”按钮在一个文本小构件上显示结果。如果文件不存在，需要在一个消息框中显示一条消息。



a)　　b)

图 14-3　程序让用户选择一个文件并显示文件中每个字母的出现次数

*14.5 （Tkinter：统计每个字母的出现次数）修改上一题显示结果的直方图，如图 14-4 所示。如果文件不存在，需要在一个消息框中显示一条消息。

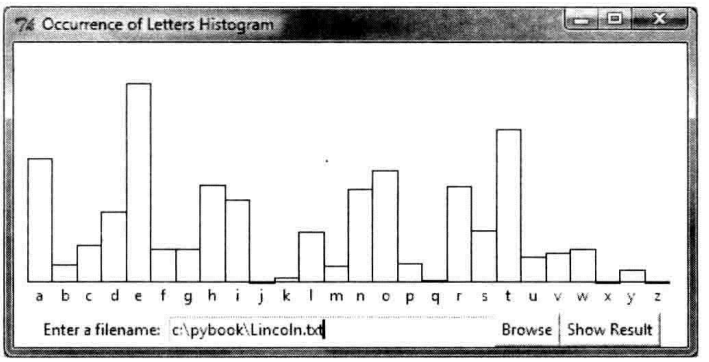


图 14-4　程序要求用户选择一个文件并将文件中每个字母的出现次数用直方图显示

*14.6 （Tkinter：统计每个字母的出现次数）重写程序清单 14-5 中的程序，使用一个 GUI 程序要求用户从一个输入域上输入 URL 地址，如图 14-5 所示。单击“Show Result”按钮在一个文本小构件上显示统计结果。如果这个 URL 地址不存在，需要在一个消息框中显示一条消息。

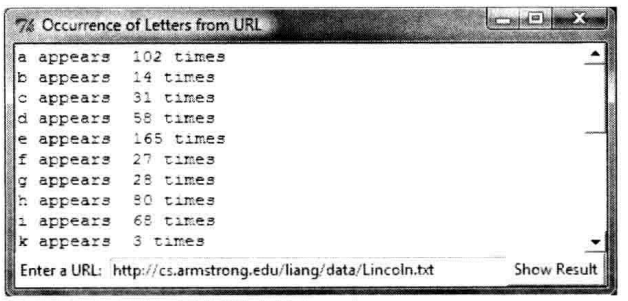


图 14-5　程序要求用户输入一个文件的 URL 地址并显示文件中每个字母的出现次数

*14.7 （Tkinter：统计每个字母的出现次数）修改上题的程序来显示结果的直方图，如图 14-6 所示。如果 URL 地址不存在，需要在一个消息框中显示一条消息。

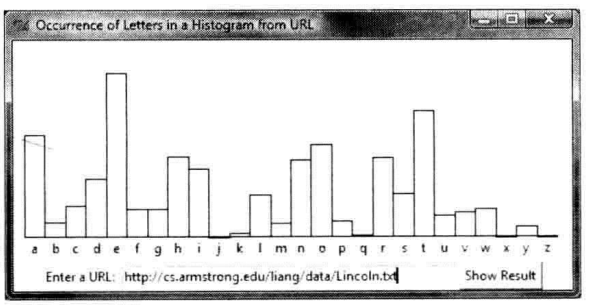


图 14-6　程序要求用户输入一个文件的 URL 地址并在一个直方图上显示文件中每个字母的出现次数

14.8 （按升序显示不重复的单词）编写一个程序提示用户输入一个文本文件，从文件中读取单词，按升序显示所有不重复的单词。

***14.9 （游戏：刽子手）编写一个刽子手游戏，使用图形进行显示，如图 14-7 所示。用户猜完七次后，程序显示这个单词。用户可以单击“Enter”键继续猜另一个单词。

*14.10 （猜首都）使用一个字典存储国家和首都对来重写编程题 11.40，这样可以随机显示问题。

*14.11 （统计辅音和元音字母）编写一个程序提示用户输入一个文本文件名并显示文件中元音字母和辅音字母的个数。使用一个集合来存储元音字母 A、E、I、O 和 U。

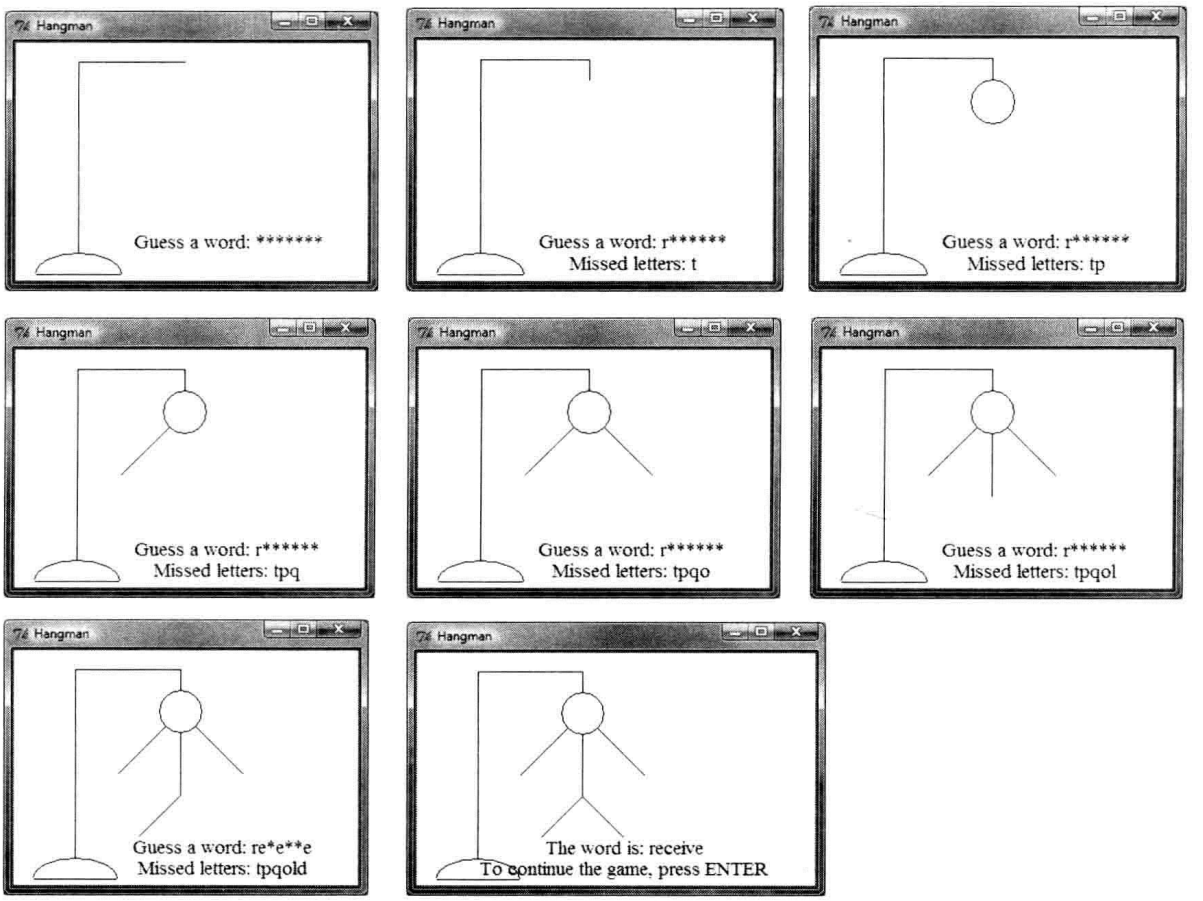


图 14-7　刽子手游戏让用户输入字母来猜单词

递　　归

学习目标

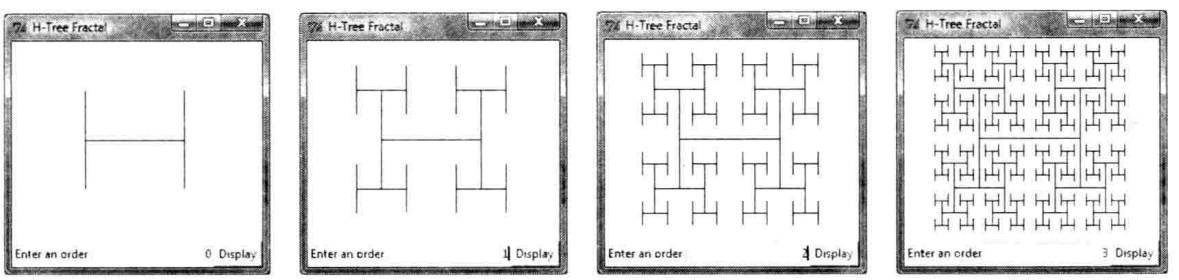
- 解释什么是递归函数并描述使用递归的好处（第 15.1 节）。
- 为递归数学函数开发递归程序（第 15.2 ～ 15.3 节）。
- 解释在调用栈中如何处理递归函数的调用（第 15.1 ～ 15.3 节）。
- 使用递归解决问题（第 15.4 节）。
- 使用辅助函数设计一个递归函数（第 15.5 节）。
- 使用递归实现选择排序（第 15.5.1 节）。
- 使用递归实现二分查找（第 15.5.2 节）。
- 使用递归获取一个目录的大小（第 15.6 节）。
- 使用递归解决汉诺塔问题（第 15.7 节）。
- 使用递归绘制分形（第 15.8 节）。
- 使用递归解决八皇后问题（第 15.9 节）。
- 了解递归和迭代之间的关联与区别（第 15.10 节）。
- 了解尾递归函数并解释为什么需要它（第 15.11 节）。

15.1　引言

关键点：*递归是一种能以优雅的解决方案来解决那些利用循环很难编程的问题的技术。*

假设希望找出某个目录下所有包含某个特定单词的文件。该如何解决这个问题呢？这里有几种方式可以解决这个问题。一个直观且有效的解决方案是使用递归在每个子目录下递归地搜索所有文件。

如图 15-1 所示的 H- 树用在超大规模集成电路（VLSI）设计上，作为一个时钟分布网络来将定时信号以相同的传播时延路由到芯片的所有部分。如何编写一个程序显示 H- 树？一个好的函数就是通过探究递归模式来使用递归。



a)　　b)　　c)　　d)

图 15-1　使用递归显示 H- 树

使用递归就是使用*递归函数*编程，递归函数就是直接或间接调用自身的函数。递归是一

种非常有用的程序设计技术。在某些情况下，对于用其他方法很难解决的问题，使用递归就能给出一个很自然、直观、简单的解决方案。本章介绍递归程序设计的概念和技术，并给出例子来演示如何“递归地思考”。

15.2　实例研究：计算阶乘

关键点：*递归函数就是一个调用自己的函数。*

许多数学函数都是使用递归来定义的。我们从一个简单的例子开始。数字 n 的阶乘可以按递归方式进行如下定义。

```
0! = 1;
n! = n × (n - 1)!; n > 0
```

对给定的 n 如何求 n! 呢？因为已经知道 0!=1，而 1!=1 × 0!，所以很容易求得 1!。假设已知 (n-1)!，使用 n!=n × (n-1)! 就可以立即得到 n!。这样，计算 n! 的问题就简化为计算 (n-1)!。当计算 (n-1)! 时，你可以递归地应用这个思路直到 n 递减为 0。

假定计算 n! 的函数是 factorial(n)。如果用 n=0 调用这个函数，立即就能返回它的结果。函数知道如何处理这个最简单的情况，这种最简单的情况被称为*基础情况*或*终止条件*。如果用 n>0 调用这个函数，它会把这个问题简化为计算 n-1 的阶乘的子问题。这个子问题在本质上和原始问题是一样的，但是它比原始问题更简单也更小。因为子问题和原始问题具有相同的性质，所以可以用不同的参数来调用这个函数，这被称作递归调用。

计算 factorial(n) 的递归算法可以简单地描述为如下方式。

```
if n == 0:
    return 1
else:
    return n * factorial(n - 1)
```

一个递归调用可能导致更多的递归调用，因为这个函数会持续地把每个子问题分解为新的子问题。要终止一个递归函数，问题必须最终递减到满足一个终止条件。当问题达到这个终止条件时，就将结果返回给调用者。然后调用者进行计算并将结果返回给它自己的调用者。这个过程持续进行，直到结果被传回原始的调用者为止。现在，原始问题就可以由 factorial(n-1) 的结果乘以 n 得到。

程序清单 15-1 是一个完整的程序，它提示用户输入一个非负整数，然后显示这个数的阶乘。

程序清单 15-1　ComputeFactorial.py

```
1  def main():
2      n = eval(input("Enter a nonnegative integer: "))
3      print("Factorial of", n, "is", factorial(n))
4
5  # Return the factorial for the specified number
6  def factorial(n):
7      if n == 0: # Base case
8          return 1
9      else:
10         return n * factorial(n - 1) # Recursive call
11
12 main() # Call the main function
```

```
Enter a nonnegative integer: 4 ↵Enter
Factorial of 4 is 24
```

```
Enter a nonnegative integer: 10 ↵Enter
Factorial of 10 is 3628800
```

本质上讲，factorial 函数（第 6 ～ 10 行）是把阶乘在数学上的递归定义直接转换为 Python 代码。因为对 factorial 的调用是调用它自己，所以这个调用是递归的。传递到 factorial 的参数一直递减，直到达到它的基础情况 0。

现在已经了解如何编写一个递归函数，下面让我们看看递归是如何工作的。图 15-2 描述了从 n = 4 开始执行的递归调用。递归调用对堆栈空间的使用如图 15-3 所示。

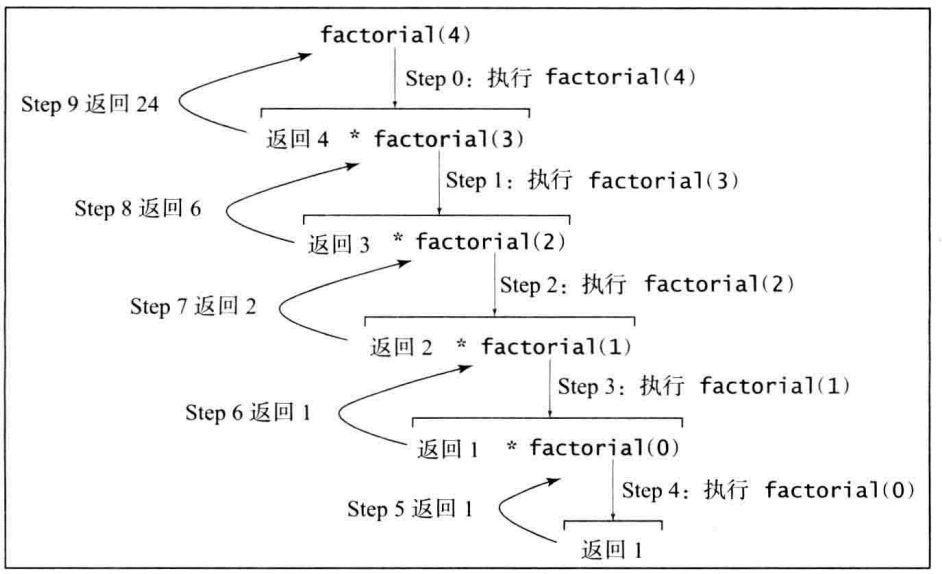


图 15-2 调用 factorial(4) 会引起对 factorial 的递归调用

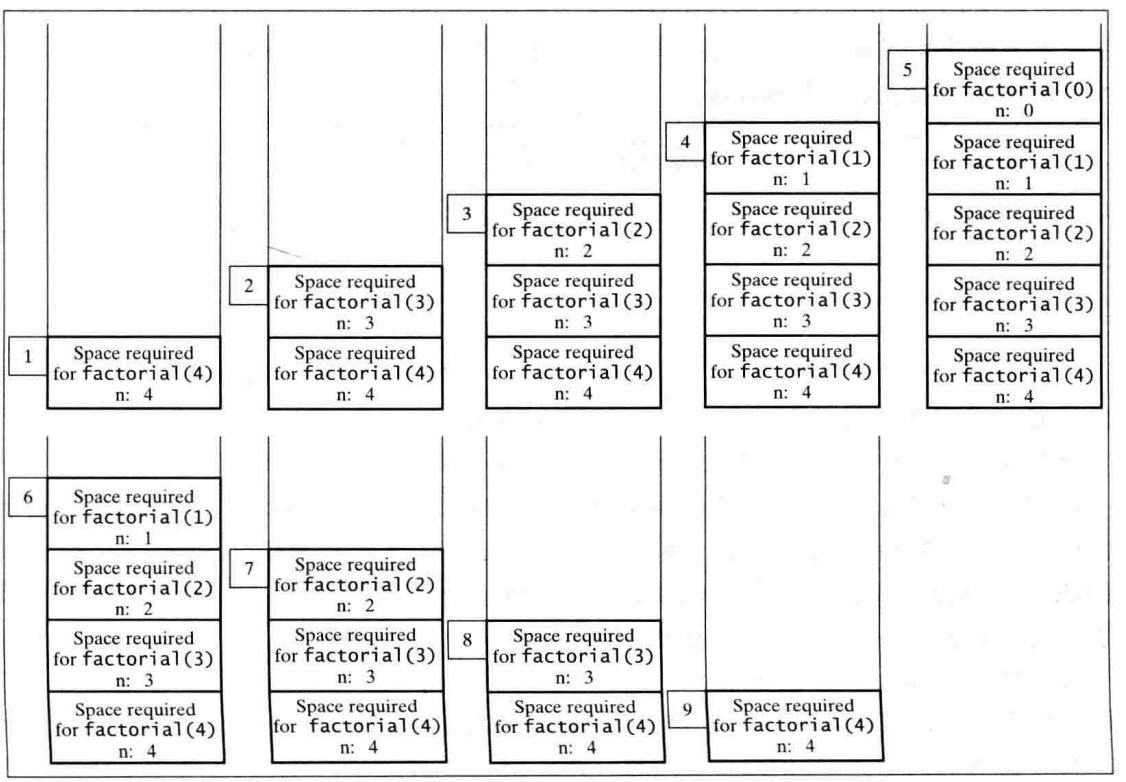


图 15-3 当执行 factorial(4) 时，factorial 函数会被递归调用，导致内存空间动态变化

教学建议：使用循环来实现 factorial 函数是一种比较简单且更加高效的方法。然而，这里使用递归的 factorial 函数是演示递归概念的一个很好的例子。在本章后续内容中，还将给出一些本质是递归且不使用递归很难解决的问题。

如果递归不能使问题简化并使之最终收敛到基础情况，就有可能出现无限递归。例如，假设错误地将 factorial 函数写成如下形式。

```
def factorial(n):
    return n * factorial(n - 1)
```

那么这个函数就会无限地运行下去，并且导致一个运行时错误。

目前讨论的例子给出一个调用自己的递归函数。这被称为直接递归。也可以创建一个间接递归。这发生在函数 A 调用函数 B，而函数 B 又反过来调用函数 A 的时候。这个递归中会涉及多个函数。例如，函数 A 调用函数 B，函数 B 调用函数 C，函数 C 调用函数 A。

检查点

15.1　递归函数是什么？

15.2　程序清单 15-1 中，如果调用 factorial(6)，那么 factorial 函数将会调用多少次？

15.3　编写一个递归的数学定义来计算 2^n，其中 n 为正整数。

15.4　编写一个递归的数学定义来计算 x^n，其中 n 为正整数，x 为实数。

15.5　编写一个递归的数学定义来计算 1 + 2 + 3 +…+ n，其中 n 为正整数。

15.6　什么是无限递归？什么是直接递归？什么是间接递归？

15.3　实例研究：计算斐波那契数

关键点：在某些情况下，递归可以给出一个自然、直接、简单的问题的解。

前一节中的 factorial 函数可以很容易地不使用递归改写。但是，在某些情况下，用其他方法不容易解决的问题可以利用递归给出一个很直观、简洁了当的解决方案。考虑众所周知的斐波那契（Fibonacci）数列问题：

```
The series: 0  1  1  2  3  5  8  13  21  34  55  89 . . .
   indexes: 0  1  2  3  4  5  6  7   8   9   10  11
```

斐波那契数列从 0 和 1 开始，之后的每个数都是序列中前两个数之和。序列可以被递归地定义为：

```
fib(0) = 0
fib(1) = 1
fib(index) = fib(index - 2) + fib(index - 1); index >= 2
```

注意：斐波那契数列是以中世纪数学家 Leonardo Fibonacci 的名字命名的，他为建立兔子繁殖数量的增长模型而构造出这个数列。这个数列可用于数值优化和其他很多领域。

对给定的 index，怎么求 fib(index) 呢？因为已知 fib(0) 和 fib(1)，所以很容易求得 fib(2)。假设已知 fib(index−2) 和 fib(index−1)，就可以立即得到 fib(index)。这样，计算 fib(index) 的问题就简化为计算 fib(index−2) 和 fib(index−1) 的问题。以这种方式求解，就可以递归地运用这个思路直到 index 递减为 0 和 1。

基础情况是 index = 0 或 index = 1。如果用 index = 0 或 index = 1 调用这个函数，它会立即返回结果。若用 index> = 2 调用这个函数，则通过使用递归调用把这个问题分解成计算 fib(index−2) 和 fib(index−1) 的两个子问题。计算 fib(index) 的递归算法可以简单地描述如下。

```
if index == 0:
    return 0
elif index == 1:
    return 1
else:
    return fib(index - 1) + fib(index - 2)
```

程序清单 15-2 给出一个完整的程序，提示用户输入一个下标，然后计算这个下标值相应的斐波那契数。

程序清单 15-2 ComputeFibonacci.py

```
1  def main():
2      index = eval(input("Enter an index for a Fibonacci number: "))
3      # Find and display the Fibonacci number
4      print("The Fibonacci number at index", index, "is", fib(index))
5
6  # The function for finding the Fibonacci number
7  def fib(index):
8      if index == 0: # Base case
9          return 0
10     elif index == 1: # Base case
11         return 1
12     else:  # Reduction and recursive calls
13         return fib(index - 1) + fib(index - 2)
14
15 main() # Call the main function
```

```
Enter an index for a Fibonacci number: 1 Enter
The Fibonacci number at index 1 is 1
```

```
Enter an index for a Fibonacci number: 6 Enter
The Fibonacci number at index 6 is 8
```

```
Enter an index for a Fibonacci number: 7 Enter
The Fibonacci number at index 7 is 13
```

程序并没有给出计算机在后台所做的大量工作。但是，图 15-4 给出为计算 fib(4) 所进行的连续递归调用。原始函数 fib(4) 产生两个递归调用：fib(3) 和 fib(2)，然后返回 fib(3)+fib(2) 的值。但是，按怎样的顺序调用这些函数呢？在 Python 中，操作数是从左到右计算的，所以在完全计算完 fib(3) 之后才会调用 fib(2)。图 15-4 中的标签表示了函数被调用的顺序。

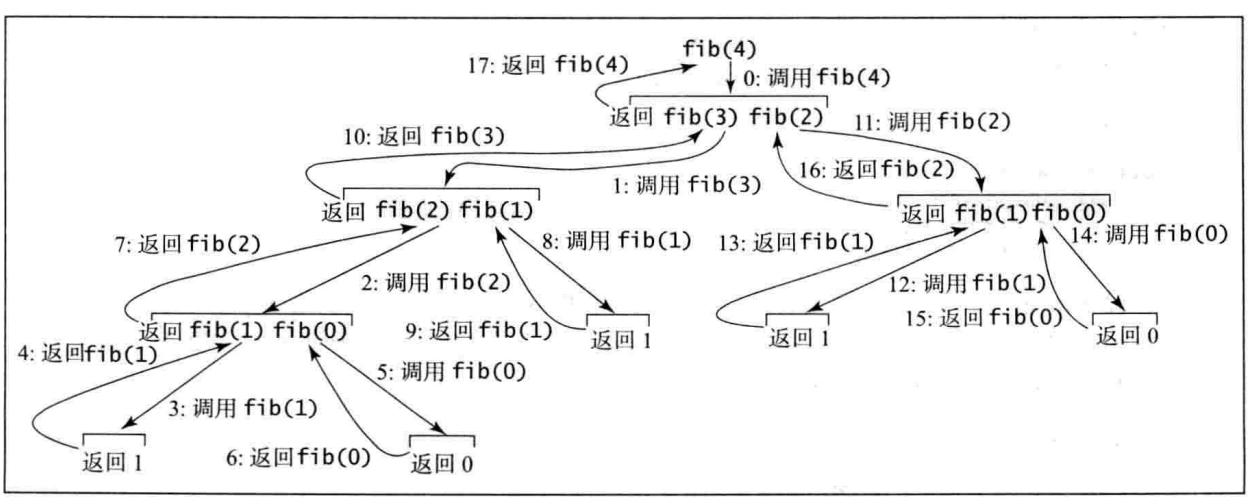


图 15-4 调用 fib(4) 会引起对 fib 的递归调用

如图 15-4 所示，这里会出现很多重复的递归调用。例如：fib(2) 被调用了 2 次，fib(1) 被调用了 3 次，fib(0) 也被调用了 2 次。通常，计算 fib(index) 所需的递归调用次数大致是计算 fib(index−1) 所需次数的 2 倍。随着尝试更大的下标值，相应的调用次数会急剧增加，如表 15-1 所示。

表 15-1　fib(n) 中的递归调用次数

n	2	3	4	10	20	30	40	50
# of calls	3	5	9	177	21 891	2 692 537	331 160 281	2 075 316 483

教学建议：fib 函数的递归实现非常简单、直接，但是并不高效。编程题 15.2 是一个使用循环的高效方案。尽管递归的 fib 函数并不实用，但它是一个演示如何编写递归函数的很好的例子。

检查点

15.7　函数 fib(6) 会调用程序清单 15-2 中的 fib 函数多少次？

15.8　给出下面程序的输出并确定它们的基础情况和递归调用。

```
def f(n):
    if n == 1:
        return 1
    else:
        return n + f(n - 1)

print("Sum is", f(5))
```

```
def f(n):
    if n > 0:
        print(n % 10)
    f(n // 10)

f(1234567)
```

15.4　使用递归解决问题

关键点：如果递归地考虑问题，许多问题都是可以使用递归来解决的。

前几节给出了两个经典的递归例子。所有的递归函数都具有以下特点：

- 函数使用 if-else 或 switch 语句会导致不同的情况。
- 一个或多个基础情况（最简单的情况）被用来停止递归。
- 每次递归调用都会简化原始问题，让它不断地接近基础情况，直到它变成基础情况为止。

通常，为了通过递归解决问题，就要将这个问题分解为许多子问题。每个子问题几乎与原始问题是一样的，只是规模小一些。可以应用相同的方法来递归地解决每个子问题。

递归无处不在。递归地思考是很有趣的。考虑喝咖啡问题。可以递归地描述这个过程，如下所示。

```
def drinkCoffee(cup):
    if cup is not empty:
        cup.takeOneSip() # Take one sip
        drinkCoffee(cup)
```

假设 cup 是一个一杯咖啡对象，它有实例函数 isEmpty() 和 takeOneSip()。可以将问题分解为两个子问题：一个是抿一小口咖啡，另一个是将杯子剩下的咖啡喝完。第二个问题与原始问题类似但是规模更小。基础情况就是当咖啡杯是空的。

考虑一条消息打印 n 次的简单问题。可以将这个问题分解为两个子问题：一个是打印消息一次，另一个是打印消息 $n-1$ 次。第二个问题与原始问题是一样的，只是规模小一些。

这个问题的基础情况是 n==0。可以使用递归来解决这个问题，如下所示。

```
def nPrintln(message, n):
    if n >= 1:
        print(message)
        nPrintln(message, n - 1)
    # The base case is n == 0
```

需要注意的是，前面一节中的 fib 函数向其调用者返回一个数值，但是 nPrintln 方法的返回类型是 void，并不向其调用者返回一个数值。

如果*递归地思考问题*，那么，本书前面章节中的许多问题都可以用递归来解决。考虑程序清单 8-1 中的回文问题。回想一下，如果一个字符串从左读和从右读是一样的，那么它就是一个回文串。例如，mom 和 dad 都是回文串，但是 uncle 和 aunt 不是回文串。确定一个字符串是否是回文串的问题可以分解为两个子问题：

- 检查字符串中的第一个字符和最后一个字符是否相等。
- 忽略两端的字符之后检查子串的其余部分是否是回文。

第二个子问题与原始问题是一样的，但是规模小一些。基础情况有两个：①两端的字符不同；②字符串大小是 0 或 1。在第一种情况下，字符串不是回文串；而在第二种情况下，字符串是回文串。这个问题的递归函数可以在程序清单 15-3 中实现。

程序清单 15-3 RecursivePalindromeUsingSubstring.py

```
1  def isPalindrome(s):
2      if len(s) <= 1: # Base case
3          return True
4      elif s[0] != s[len(s) - 1]: # Base case
5          return False
6      else:
7          return isPalindrome(s[1 : len(s) - 1])
8
9  def main():
10     print("Is moon a palindrome?", isPalindrome("moon"))
11     print("Is noon a palindrome?", isPalindrome("noon"))
12     print("Is a a palindrome?", isPalindrome("a"))
13     print("Is aba a palindrome?", isPalindrome("aba"))
14     print("Is ab a palindrome?", isPalindrome("ab"))
15
16 main() # Call the main function
```

```
Is moon a palindrome? False
Is noon a palindrome? True
Is a a palindrome? True
Is aba a palindrome? True
Is ab a palindrome? False
```

第 7 行的字符串切片运算符创建了一个新字符串，它除了没有原始字符串中的第一个和最后一个字符，其余都和原始字符串一样。如果原始字符串两端字符相同，那么检查一个字符串是否是回文与检查子串是否是回文的函数是一样的。

检查点

15.9 描述递归函数的特点？

15.10 使用程序清单 15-3 中定义的函数给出 isPalinedrome("abcba") 的调用栈。

15.11 给出下面两个程序的输出。

```
def f(n):
    if n > 0:
        print(n, end = ' ')
        f(n - 1)

f(5)
```

```
def f(n):
    if n > 0:
        f(n - 1)
        print(n, end = ' ')

f(5)
```

15.12　下面函数中哪里有错误？

```
def f(n):
    if n != 0:
        print(n, end = ' ')
        f(n / 10)

f(1234567)
```

15.5　递归辅助函数

关键点：有时可以通过轻微地改变原始问题找出一个递归的解决方案。这个新函数被称为递归辅助函数。可以调用递归辅助函数来解原始问题。

因为前面递归的 isPalindrome 函数要为每次递归调用创建一个新字符串，因此它不够高效。为避免创建新字符串，可以使用 low 和 high 下标来表明子串的范围。这两个下标必须传递给递归函数。由于原始函数是 isPalindrome(String s)，因此，必须创建一个新函数 isPalindrome Helper (String s，int low，int high) 来接收字符串的附加信息，如程序清单 15-4 所示。

程序清单 15-4　RecursivePalindrome.py

```
1  def isPalindrome(s):
2      return isPalindromeHelper(s, 0, len(s) - 1)
3
4  def isPalindromeHelper(s, low, high):
5      if high <= low: # Base case
6          return True
7      elif s[low] != s[high]: # Base case
8          return False
9      else:
10         return isPalindromeHelper(s, low + 1, high - 1)
11
12 def main():
13     print("Is moon a palindrome?", isPalindrome("moon"))
14     print("Is noon a palindrome?", isPalindrome("noon"))
15     print("Is a a palindrome?", isPalindrome("a"))
16     print("Is aba a palindrome?", isPalindrome("aba"))
17     print("Is ab a palindrome?", isPalindrome("ab"))
18
19 main() # Call the main function
```

函数 isPalindrome(s) 检查一个字符串 s 是否是一个回文串，而函数 isPalindrome-Helper(s, low, high) 检查一个子串 s[low:high+1] 是否是一个回文串。函数 isPalindrome(s) 将 low = 0 和 high = len(s)−1 的字符串 s 传递给函数 isPalindromeHelper。函数 isPalindrome-Helper 可以被递归地调用，以检查不断缩减的子串是否是回文串。在递归程序设计中定义第二个函数来接收附加参数是一个常用的设计技巧。这样的函数被称为*递归辅助函数*。

辅助函数在设计关于字符串和数组问题的递归解决方案上是非常有用的。下面几节将给

出两个以上的例子。

15.5.1 选择排序

选择排序在第 10.11.1 节中已经介绍过。回顾一下，选择排序法是指先找出列表中的最小元素，并将它和第一个元素互换。然后，在剩余的元素中找出最小数，再将它和剩余列表中的第一个元素互换，这样的过程一直进行下去，直到列表中仅剩一个元素为止。这个问题可以被分解为两个子问题：

- 找出列表中的最小元素，然后将它与第一个元素交换。
- 忽略第一个元素，对剩余的较小列表进行递归排序。

基础情况是该列表只包含一个元素。程序清单 15-5 给出了递归的排序函数。

程序清单 15-5 RecursiveSelectionSort.py

```
def sort(lst):
    sortHelper(lst, 0, len(lst) - 1) # Sort the entire list

def sortHelper(lst, low, high):
    if low < high:
        # Find the smallest element and its index in lst[low .. high]
        indexOfMin = low
        min = lst[low]
        for i in range(low + 1, high + 1):
            if lst[i] < min:
                min = lst[i]
                indexOfMin = i

        # Swap the smallest in lst[low .. high] with lst[low]
        lst[indexOfMin] = lst[low]
        lst[low] = min

        # Sort the remaining lst[low+1 .. high]
        sortHelper(lst, low + 1, high)

def main():
    lst = [3, 2, 1, 5, 9, 0]
    sort(lst)
    print(lst)

main() # Call the main function
```

函数 sort(lst) 对列表 lst[0..len(lst)-1] 进行排序，而函数 sortHelper(lst，low，high) 是对子列表 lst[low..high] 进行排序。第二个函数被递归地调用，以对不断缩小的子列表进行排序。

15.5.2 二分查找

二分查找已经在第 10.10.2 节中介绍过。使用二分查找法的前提条件是数组元素必须已经排好序。二分查找法首先将关键字与数组的中间元素进行比较。考虑下面三种情况。

- 情况 1：如果关键字比中间元素小，那么程序只需在列表的前半段中进行递归查找。
- 情况 2：如果关键字和中间元素相等，则匹配成功，查找结束。
- 情况 3：如果关键字比中间元素大，那么程序只需在列表的后半段中进行递归查找。

情况 1 和情况 3 都将查找范围降为一个更小的数列。当匹配成功时，情况 2 就是一个基础情况。另一个基础情况是查找完毕而没有一个成功的匹配。程序清单 15-6 使用递归给二分查找问题提供一个清晰、简单的解决方案。

程序清单 15-6 RecursiveBinarySearch.py

```
1  def recursiveBinarySearch(lst, key):
2      low = 0
3      high = len(lst) - 1
4      return recursiveBinarySearchHelper(lst, key, low, high)
5
6  def recursiveBinarySearchHelper(lst, key, low, high):
7      if low > high:  # The list has been exhausted without a match
8          return -low - 1
9
10     mid = (low + high) // 2
11     if key < lst[mid]:
12         return recursiveBinarySearchHelper(lst, key, low, mid - 1)
13     elif key == lst[mid]:
14         return mid
15     else:
16         return recursiveBinarySearchHelper(lst, key, mid + 1, high)
17
18 def main():
19     lst = [3, 5, 6, 8, 9, 12, 34, 36]
20     print(recursiveBinarySearch(lst, 3))
21     print(recursiveBinarySearch(lst, 4))
22
23 main() # Call the main function
```

函数 recursiveBinarySearch 是在整个列表中查找关键字（第 1 ～ 4 行）。而函数 recursiveBinarySearchHelper 是在列表中从下标 low 到 high 中查找关键字（第 6 ～ 16 行）。

函数 recursiveBinarySearch 将 low=0（第 2 行）和 high=len(lst)−1（第 3 行）的初始列表传递给函数 recursiveBinarySearchHelper。函数 recursiveBinarySearchHelper 被递归地调用，在一个不断缩小的子列表中查找关键字。

检查点

15.13　什么是一个递归辅助函数？

15.14　使用程序清单 15-5 中定义的函数，给出 sort([2，3，5，1]) 的调用栈。

15.6　实例研究：求出目录的大小

关键点：递归函数可以使用递归结构高效地解决这个问题。

前面的例子可以不用递归很容易地解决。本节给出的这个问题，要是不使用递归是很难解决的。这里的问题是求出一个目录的大小。一个目录的大小是指该目录下所有文件大小之和。目录 d 可能会包含子目录。假设一个目录包含文件 f_1，f_2，…，f_m 以及子目录 d_1，d_2，…，d_n，如图 15-5 所示。

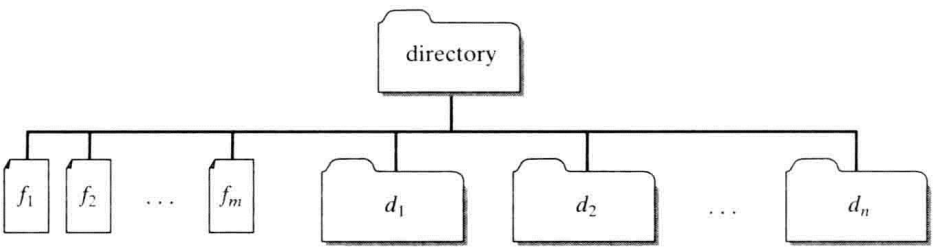


图 15-5　一个目录包含多个文件和子目录

目录的大小可以按如下方式递归地定义。

$$size(d) = size(f_1) + size(f_2) + \ldots + size(f_m) + size(d_1) + size(d_2) + \ldots + size(d_n)$$

为了实现这个程序，需要 os 模块中的以下三个函数：

- os.path.isfile(f)，如果 s 是一个文件名则返回 True。回顾一下，在第 13.2.3 节中介绍这个函数来检测一个文件是否存在。
- os.path.getsize(filename) 返回这个文件的大小。
- os.listdir(directory) 返回一个子目录列表以及目录下的文件。

程序清单 15-7 中的程序提示用户输入一个目录或一个文件，然后显示它的大小。

程序清单 15-7 DirectorySize.py

```
1  import os
2
3  def main():
4      # Prompt the user to enter a directory or a file
5      path = input("Enter a directory or a file: ").strip()
6
7      # Display the size
8      try:
9          print(getSize(path), "bytes")
10     except:
11         print("Directory or file does not exist")
12
13 def getSize(path):
14     size = 0 # Store the total size of all files
15
16     if not os.path.isfile(path):
17         lst = os.listdir(path) # All files and subdirectories
18         for subdirectory in lst:
19             size += getSize(path + "\\" + subdirectory)
20     else: # Base case, it is a file
21         size += os.path.getsize(path) # Accumulate file size
22
23     return size
24
25 main() # Call the main function
```

```
Enter a directory or a file: c:\pybook Enter
619631 bytes
```

```
Enter a directory or a file: c:\pybook\Welcome.py Enter
76 bytes
```

```
Enter a directory or a file: c:\book\NonExistentFile Enter
Directory or file does not exist
```

如果 path 是一个目录（第 16 行），那么该目录下的每个子条目（文件或子目录）都被递归地调用以获取它的大小（第 19 行）。如果 path 是一个文件（第 20 行），获取的就是该文件的大小（第 21 行）。

如果用户输入的是一个错误的目录或者不存在的目录，程序将会抛出一个异常（第 11 行）。

提示：为了避免错误，测试基础情况是一个很好的办法。例如：应该输入一个文件、一个空目录、一个不存在的目录以及一个不存在的文件来测试这个程序。

检查点

15.15 使用什么函数测试一个文件是否存在？使用什么函数返回一个文件的大小？使用什么函数返回一个目录下的所有文件和子目录？

15.7　实例研究：汉诺塔

关键点：汉诺塔问题是一个经典的问题，它可以使用递归很容易地解决，但是，不使用递归则非常难解决。

汉诺塔问题是一个几乎每一个计算机科学家都知道的经典的递归问题。这个问题是将指定个数而大小互不相同的盘子从一个塔上移到另一个塔上，移动要遵从下面的规则：

- n 个标记 1、2、3、…、n 的盘子，以及三个标记 A、B、C 的塔。
- 任何时候盘子都不能放在比它小的盘子的上方。
- 初始状态时，所有的盘子都被放在塔 A 上。
- 每次只能移动一个盘子，并且这个盘子必须在塔顶位置。

这个问题的目标是借助塔 C 把所有的盘子从塔 A 移到塔 B。例如：如果有三个盘子，将所有的盘子从塔 A 移到塔 B 的步骤如图 15-6 所示。

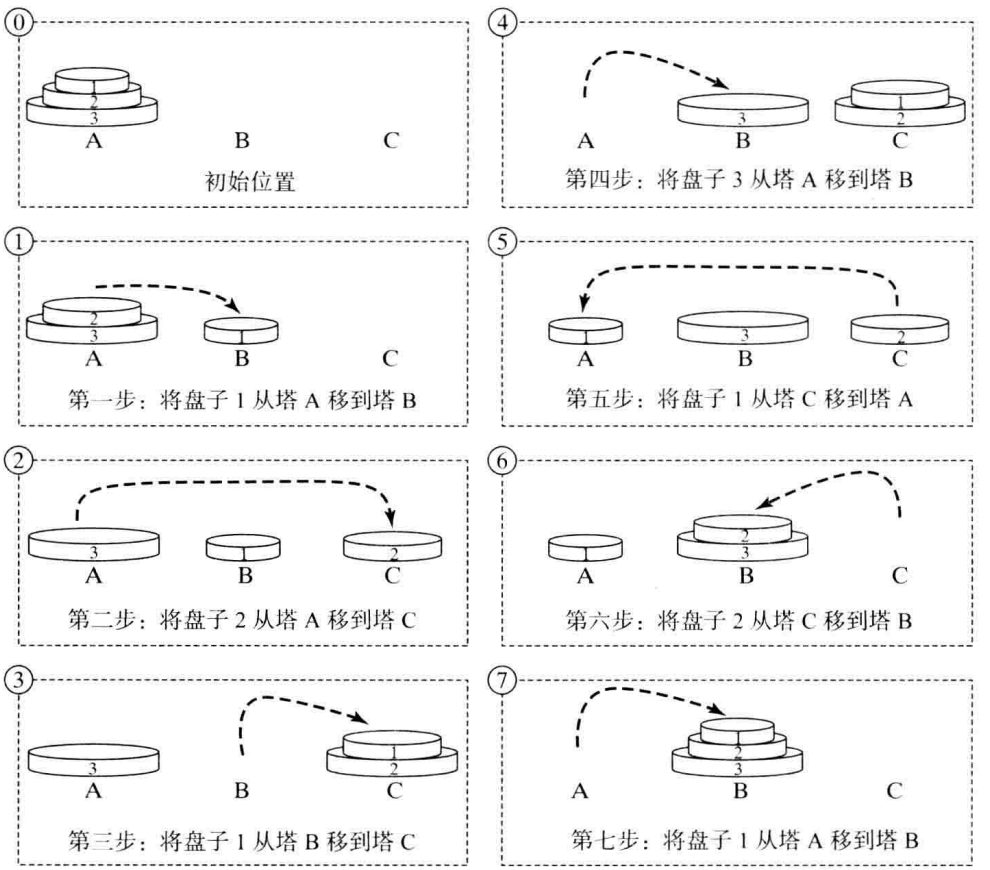


图 15-6　汉诺塔问题的目的是在遵从规则的条件下把盘子从塔 A 移到塔 B

注意：汉诺塔是一个经典的计算机科学问题。许多网站都有关于该问题的解法。其中一个很值得一看的网站是 www.cut-the-knot.com/recurrence/hanoi.shtml。

在三个盘子的情况下，可以手动地找出解决方案。然而，当盘子数量较大时，即使是 4 个，这个问题还是非常复杂的。幸运的是，这个问题本身就具有递归性质，可以直接得到直观的递归解决方案。

这个问题的基础情况是 n = 1。若 n = 1，你就可以简单地把盘子从塔 A 移到塔 B。当

n>1 时，可以将原始问题拆分成下面的三个子问题，然后依次解决，如下所示：

1）借助塔 B 将前 n−1 个盘子从 A 移到 C，如图 15-7 中的步骤 1 所示。

2）将盘子 n 从塔 A 移到塔 B，如图 15-7 中的步骤 2 所示。

3）借助塔 A 将 n−1 个盘子从塔 C 移到塔 B，如图 15-7 中的步骤 3 所示。

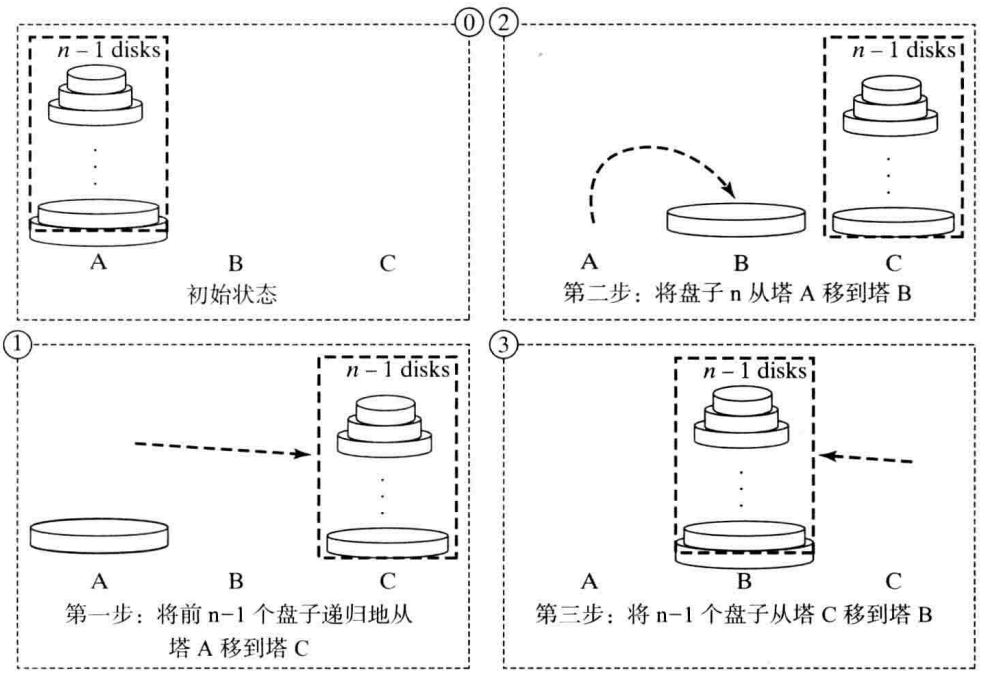


图 15-7 汉诺塔问题可以被分解成三个子问题

下面的函数借助辅助塔 auxTower 将 n 个盘子从原始塔 fromTower 移到目标塔 toTower：

```
def moveDisks(n, fromTower, toTower, auxTower):
```

这个函数的算法可以如下描述。

```
if n == 1: # Stopping condition
    Move disk 1 from the fromTower to the toTower
else:
    moveDisks(n - 1, fromTower, auxTower, toTower)
    Move disk n from the fromTower to the toTower
    moveDisks(n - 1, auxTower, toTower, fromTower)
```

程序清单 15-8 中的程序提示用户输入盘子个数，然后调用递归函数 moveDisks 来显示移动盘子的解决方案。

程序清单 15-8 TowersOfHanoi.py

```
1  def main():
2      n = eval(input("Enter number of disks: "))
3
4      # Find the solution recursively
5      print("The moves are:")
6      moveDisks(n, 'A', 'B', 'C')
7
8  # The function for finding the solution to move n disks
9  #   from fromTower to toTower with auxTower
10 def moveDisks(n, fromTower, toTower, auxTower):
11     if n == 1: # Stopping condition
```

```
12          print("Move disk", n, "from", fromTower, "to", toTower)
13      else:
14          moveDisks(n - 1, fromTower, auxTower, toTower)
15          print("Move disk", n, "from", fromTower, "to", toTower)
16          moveDisks(n - 1, auxTower, toTower, fromTower)
17
18  main() # Call the main function
```

```
Enter number of disks: 4 ↵Enter
The moves are:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Move disk 4 from A to B
Move disk 1 from C to B
Move disk 2 from C to A
Move disk 1 from B to A
Move disk 3 from C to B
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
```

考虑跟踪 n = 3 的程序。连续的递归调用如图 15-8 所示。正如你所见，编写这个程序比跟踪这个递归调用要容易些。系统使用栈来跟踪后台的调用。从某种程度上讲，递归提供了某种层次的抽象，这种抽象对用户隐藏迭代和其他细节。

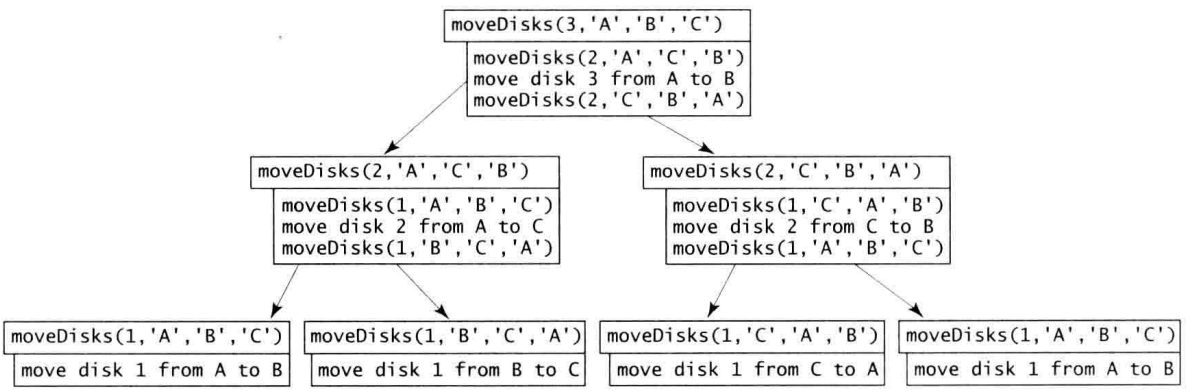


图 15-8　调用 moveDisks（3，'A'，'B'，'C'）会引起对 moveDisks 的递归调用

检查点

15.16　为了调用 moveDisks（5，'A'，'B'，'C'），会调用多少次程序清单 15-8 中的 moveDisks 函数？

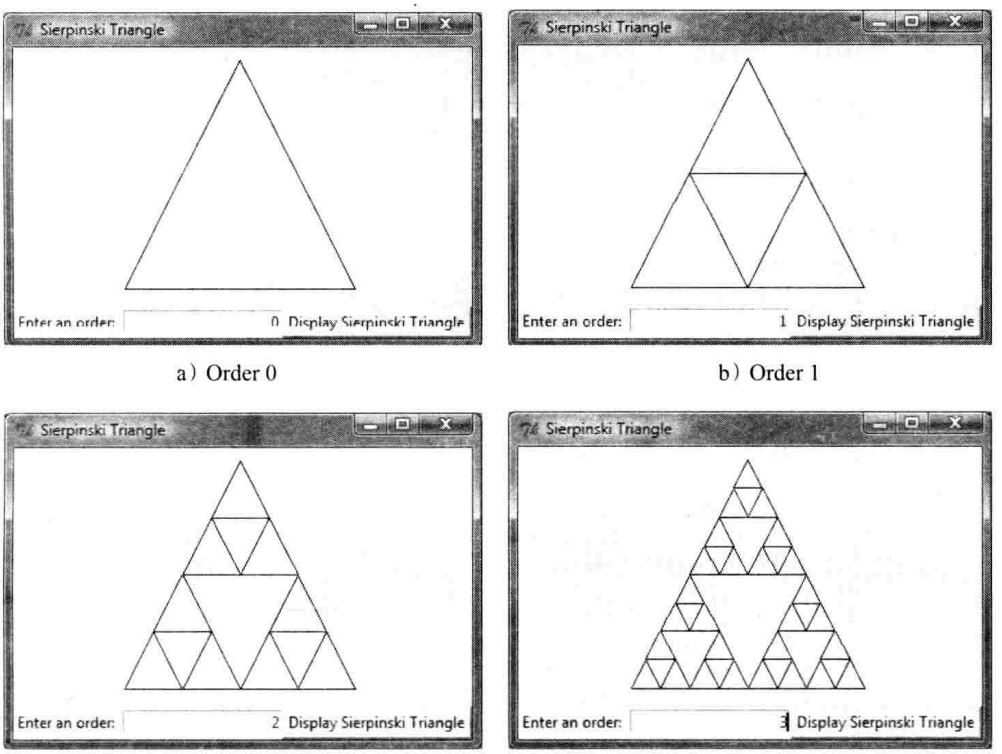
15.8　实例研究：分形

关键点：*递归是显示分形的理想方法，因为分形本质上就是递归。*

分形是一个几何图形，但是它不像三角形、圆形和矩形。分形可以被分为几个部分，每部分都是整体的一个缩小版本。分形有许多有趣的例子。本节介绍一个被称为谢尔宾斯基三角形（sierpinski triangle）的简单分形，它是以一位著名的波兰数学家的名字来命名的。

谢尔宾斯基三角形是按如下方式创建的：

- 从一个等边三角形开始，将它作为 0 阶（或 0 级）的谢尔宾斯基分形，如图 15-9a 所示。
- 将 0 阶三角形的各边中点连接起来产生 1 阶谢尔宾斯基三角形（图 15-9b）。
- 保持中间的三角形不变，将另外三个三角形各边的中点连接起来产生 2 阶谢尔宾斯基分形（图 15-9c）。
- 可以递归地重复同样的步骤产生 3 阶、4 阶、……、n 阶的谢尔宾斯基三角形（图 15-9d）。



a）Order 0　　b）Order 1

c）Order 2　　d）Order 3

图 15-9　谢尔宾斯基三角形是一种递归三角形的图形

这个问题本质上是递归的。那么，该如何开发针对该问题的递归方案呢？考虑阶数为 0 的基础情况。绘制出 0 阶谢尔宾斯基三角形是很容易的。那么如何绘制出 1 阶谢尔宾斯基三角形呢？这个问题可以被简化为绘制三个 0 阶谢尔宾斯基三角形。如何绘制 2 阶谢尔宾斯基三角形呢？这个问题可以被简化为绘制三个 1 阶谢尔宾斯基三角形。因此，绘制 n 阶谢尔宾斯基三角形可以简化为绘制三个 n−1 阶谢尔宾斯基三角形。

程序清单 15-9 给出显示任意阶谢尔宾斯基三角形的程序，如图 15-9 所示。可以在文本域输入阶数，然后显示这个指定阶数的谢尔宾斯基三角形。

程序清单 15-9 SierpinskiTriangle.py

```
1  from tkinter import * # Import all definition from tkinter
2
3  class SierpinskiTriangle:
4      def __init__(self):
5          window = Tk() # Create a window
6          window.title("Sierpinski Triangle") # Set a title
7
8          self.width = 200
9          self.height = 200
10         self.canvas = Canvas(window,
```

```
11              width = self.width, height = self.height)
12          self.canvas.pack()
13
14          # Add a label, an entry, and a button to frame1
15          frame1 = Frame(window) # Create and add a frame to window
16          frame1.pack()
17
18          Label(frame1,
19              text = "Enter an order: ").pack(side = LEFT)
20          self.order = StringVar()
21          entry = Entry(frame1, textvariable = self.order,
22                        justify = RIGHT).pack(side = LEFT)
23          Button(frame1, text = "Display Sierpinski Triangle",
24              command = self.display).pack(side = LEFT)
25
26          window.mainloop() # Create an event loop
27
28      def display(self):
29          self.canvas.delete("line")
30          p1 = [self.width / 2, 10]
31          p2 = [10, self.height - 10]
32          p3 = [self.width - 10, self.height - 10]
33          self.displayTriangles(int(self.order.get()), p1, p2, p3)
34
35      def displayTriangles(self, order, p1, p2, p3):
36          if order == 0: # Base condition
37              # Draw a triangle to connect three points
38              self.drawLine(p1, p2)
39              self.drawLine(p2, p3)
40              self.drawLine(p3, p1)
41          else:
42              # Get the midpoint of each triangle's edge
43              p12 = self.midpoint(p1, p2)
44              p23 = self.midpoint(p2, p3)
45              p31 = self.midpoint(p3, p1)
46
47              # Recursively display three triangles
48              self.displayTriangles(order - 1, p1, p12, p31)
49              self.displayTriangles(order - 1, p12, p2, p23)
50              self.displayTriangles(order - 1, p31, p23, p3)
51
52      def drawLine(self, p1, p2):
53          self.canvas.create_line(
54              p1[0], p1[1], p2[0], p2[1], tags = "line")
55
56      # Return the midpoint between two points
57      def midpoint(self, p1, p2):
58          p = 2 * [0]
59          p[0] = (p1[0] + p2[0]) / 2
60          p[1] = (p1[1] + p2[1]) / 2
61          return p
62
63  SierpinskiTriangle() # Create GUI
```

当在文本域中输入一个阶数然后单击“Display Sierpinski Triangle”按钮后，回调函数display被调用来创建三个点并显示这个三角形（第30～33行）。

三角形的这三个点被传递以调用displayTriangles函数（第35行）。如果阶数order==0，那么displayTriangles（order, p1, p2, p3）函数将显示一个连接三点p1、p2和p3的三角形（第38～40行），如图15-10a所示。否则，完成下面的任务。

1）获取p1和p2的中点（第43行），p2和p3的中点（第44行）以及p3和p1的中点

（第 45 行），如图 15-10b 所示。

2）使用递减的阶数递归地调用 displayTriangles，显示三个更小的谢尔宾斯基三角形（第 48 ～ 50 行）。注意：每个小的谢尔宾斯基三角形除了阶数会少一个之外，其结构和原始的大谢尔宾斯基三角形是一样的，如图 15-10b 所示。

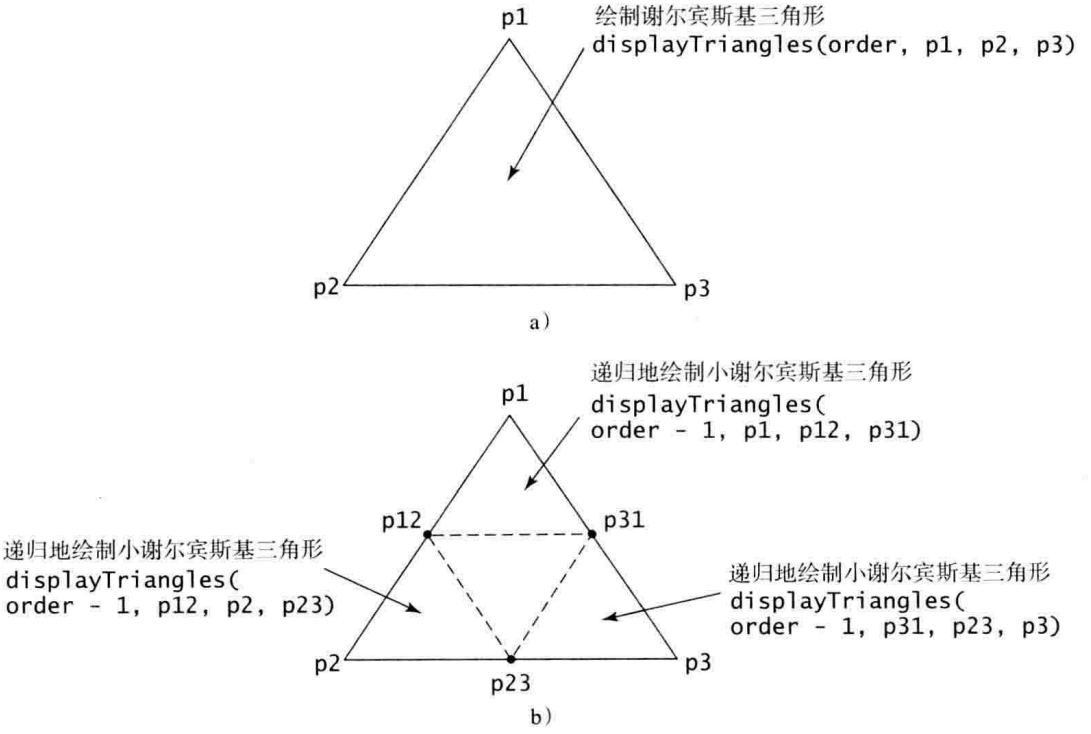


图 15-10 绘制一个谢尔宾斯基三角形会引起对绘制三个小谢尔宾斯基三角形的调用

15.9 实例研究：八皇后

关键点：八皇后问题是找出在棋盘上每行放一个皇后且不出现两个皇后互相攻击的解决方案。

本实例创建一个程序来安排棋盘上的八皇后。棋盘上的每一行只能放一个皇后，且必须放置在不出现两个皇后互相攻击的位置。需要使用一个二维列表来表示一个棋盘，但是，因为每行都只能有一个皇后，所以，使用一个一维列表来表示皇后在行上的位置就足够了。因此，如下创建名为 queens 的列表：

```
queens = 8 * [-1]
```

将 j 赋值给 queens[i] 表示皇后被放置在第 i 行第 j 列。图 15-11a 给出图 15-11b 中所示棋盘对应的列表 queens 的内容。初始状态时，queens[i] = −1 表示第 i 行没有被占用。

程序清单 15-10 中的程序显示了八皇后问题的一个解法。

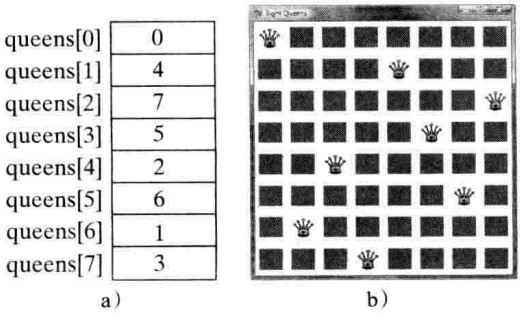


图 15-11 queens[i] 表示第 i 行皇后的位置

程序清单 15-10 EightQueens.py

```
1  from tkinter import * # Import all definitions from tkinter
2
3  SIZE = 8 # The size of the chessboard
4  class EightQueens:
5      def __init__(self):
6          self.queens = SIZE * [-1] # Queen positions
7          self.search(0) # Search for a solution from row 0
8
9          # Display solution in queens
10         window = Tk() # Create a window
11         window.title("Eight Queens") # Set a title
12
13         image = PhotoImage(file = "image/queen.gif")
14         for i in range(SIZE):
15             for j in range(SIZE):
16                 if self.queens[i] == j:
17                     Label(window, image = image).grid(
18                         row = i, column = j)
19                 else:
20                     Label(window, width = 5, height = 2,
21                         bg = "red").grid(row = i, column = j)
22
23         window.mainloop() # Create an event loop
24
25     # Search for a solution starting from a specified row
26     def search(self, row):
27         if row == SIZE: # Stopping condition
28             return True # A solution found to place 8 queens
29
30         for column in range(SIZE):
31             self.queens[row] = column # Place it at (row, column)
32             if self.isValid(row, column) and self.search(row + 1):
33                 return True # Found and exit for loop
34
35         # No solution for a queen placed at any column of this row
36         return False
37
38     # Check if a queen can be placed at row i and column j
39     def isValid(self, row, column):
40         for i in range(1, row + 1):
41             if (self.queens[row - i] == column # Check column
42                 or self.queens[row - i] == column - i
43                 or self.queens[row - i] == column + i):
44                 return False # There is a conflict
45         return True # No conflict
46
47 EightQueens() # Create GUI
```

程序初始化列表 queens 为 8 个 -1 值来表明此时没有皇后被放置在棋盘上（第 6 行）。程序调用 search(0)（第 7 行）来启动一个从第 0 行开始的解决方案，它递归地调用 search(1)、search(2)、…、search(7)（第 32 行）。

在找到一个解决方案之后，程序在窗口中显示 64 个标签（每行 8 个）并将一个皇后图像放在每行 queen[i] 对应的单元（第 17 行）。

如果所有的行都被填满，那么递归的 search(row) 函数会返回 True（第 27 ～ 28 行）。该函数在一个 for 循环中检测一个皇后是否放置在第 0 列、第 1 列、第 2 列、…、第 7 列中（第 30 行）。将一个皇后放置在某一列中（第 31 行）。如果这种放法是合法的，那么调用 search(row+1) 递归地查找下一行（第 32 行）。如果查找是成功的，返回 True 并退出这个 for

循环（第 33 行）。在这种情况下，无须查找这行的下一列。如果没有将一个皇后放置在这一行的任意一列的解决方案，这个方法返回 False（第 36 行）。

假设调用行 row 为 3 的 search(row)，如图 15-12a 所示。这个函数会以第 0 列、第 1 列、第 2 列、……这样的顺序填充一个皇后。对于每次尝试，调用 isValid(row, column) 函数（第 32 行）来检测将一个皇后放在指定的位置是否会引起和之前放置的皇后的冲突。它确保没有皇后被放在同一列（第 41 行），没有皇后被放置在左上对角线上（第 42 行），没有皇后被放置在右上对角线上（第 43 行），如图 15-12b 所示。如果 isValid(row, column) 返回 False，程序就检查下一列，如图 15-12c 所示。如果 isValid(row, column) 返回 True，程序就递归地调用 search(row+1)，如图 15-12d 所示。如果 search(row+1) 返回 False，程序就检查前一行的下一列，如图 15-12c 所示。

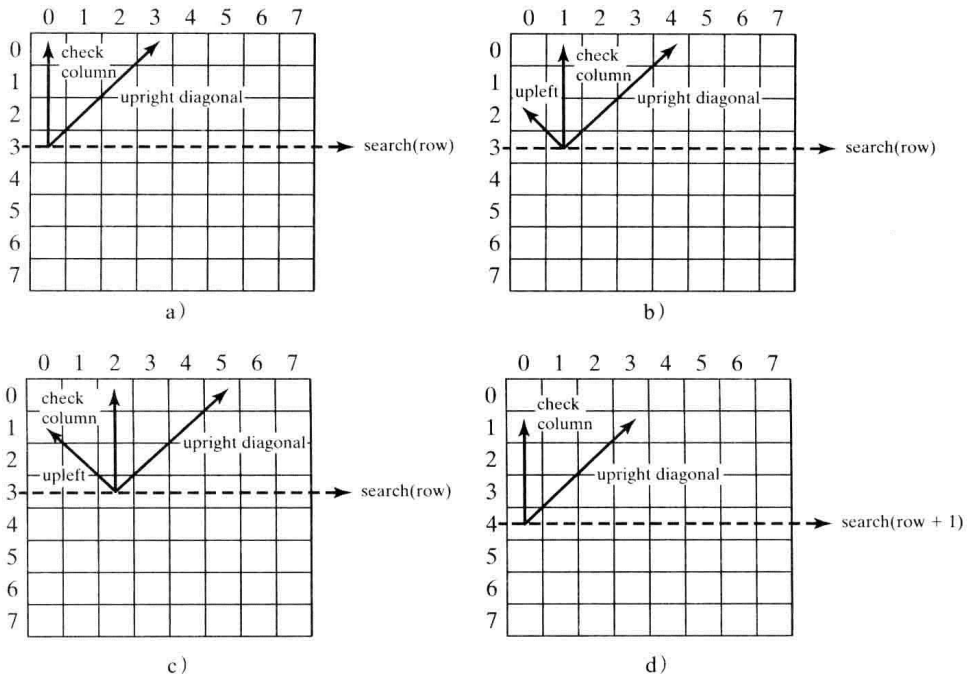


图 15-12 调用 search(row) 在某行的一列上填充皇后

15.10 递归和迭代

关键点：*递归是程序控制的一种可替代方式，它实质上就是不用循环控制的重复。*

使用循环时，指定一个循环体。循环体的重复是被循环控制结构所控制的。在递归中，函数重复地调用它自己。必须使用一条选择语句来控制是否继续递归调用该函数。

递归会产生相当大的开销。程序每调用一个函数，系统就必须给函数所有的局部变量和参数分配空间。这就要占用大量的内存，还需要额外的时间来管理这些附加的内存空间。

任何使用递归解决的问题都可以用迭代非递归地解决。递归至少会有一个副作用：它耗费了太多的时间并占用了太多的内存。那么，为什么还要用它呢？因为在某些情况下，本质上有递归特性的问题很难用其他方法解决，而递归可以给出一个清晰、简单的解决方案。例如：目录大小问题、汉诺塔问题和分形问题都是不使用递归就很难解决的问题。

应该根据要解决的问题的本质和我们对这个问题的理解来决定是用递归还是用迭代。根

据经验，选择使用递归还是迭代的原则，就是看它能否给出一个反映问题本质的直观解法。如果迭代的解决方案是显而易见的，那就使用迭代。迭代通常比选择递归效率更高。

注意：递归的程序可能会用完内存，引起一个栈溢出错误。

提示：如果关注程序的性能，就要避免使用递归，因为它会比迭代占用更多的时间且消耗更多的内存。通常，递归可以用来解决本质上是递归的问题，例如：汉诺塔、目录大小以及谢尔宾斯基三角形。

检查点

15.17　下面哪个陈述是正确的？

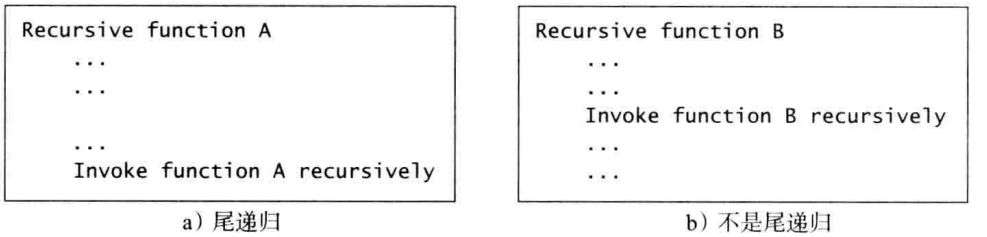
- 任何一个递归函数都可以被转换为非递归函数。
- 递归函数会比非递归函数占用更多的时间和内存。
- 递归函数总是比非递归函数更简单。
- 递归函数中总是有一条选择语句来检查是否到达基础情况。

15.18　引起一个栈溢出异常的原因是什么？

15.11　尾递归

关键点：尾递归对减少堆栈大小很有效。

如果从递归调用返回时没有待处理操作要完成，那么这个递归函数就被称为尾递归，如图 15-13a 所示。但是，图 15-13b 中的函数 B 不是尾递归，因为从每个函数调用返回时都有待处理操作要完成。



a）尾递归　　b）不是尾递归

图　15-13

例如，因为在程序清单 15-4 中的第 10 行递归调用 isPalindromeHelper 之后没有待处理的操作，所以，递归的 isPalindrome Helper 函数（第 4 ～ 10 行）就是尾递归的。但是，在程序清单 15-1 中，因为从每个递归调用返回时都有一个待处理的操作，即乘法要完成，所以，递归的 factorial 函数（第 6 ～ 10 行）就不是尾递归的。

尾递归是很必要的：因为最后一个递归调用结束时，函数也结束了，因此，无须将中间调用存储在栈中。

通常，可以使用辅助参数将非尾递归函数转换为递归函数。使用这些参数来放置结果，思路是将待处理的操作和辅助参数以一种递归调用不再有待处理操作的形式相结合。可能会定义一个带辅助参数的新的辅助递归函数。例如：程序清单 15-1 中的 factorial 函数可以被改写成尾递归形式，如下所示。

```
1  # Return the factorial for a specified number
2  def factorial(n):
3      return factorialHelper(n, 1) # Call auxiliary function
4
```

```
5  # Auxiliary tail-recursive function for factorial
6  def factorialHelper(n, result):
7      if n == 0:
8          return result
9      else:
10         return factorialHelper(n - 1, n * result)
```

第一个 factorial 函数只是简单调用了辅助函数（第 3 行）。在第 6 行，辅助函数包括了辅助参数 result，它存储了 n 的阶乘的结果。这个函数在第 10 行被递归地调用。在调用返回之后，就没有了待处理的操作。最终的结果在第 8 行返回，它也是在第 3 行调用 factorial(n, 1) 的返回值。

检查点

15.19 什么是尾递归？

15.20 为什么尾递归是可描述的？

15.21 程序清单 15-5 中的递归选择函数是尾递归吗？

15.22 使用尾递归重写程序清单 15-2 中的 fib 函数。

关键术语

base case（基础情况）
direct recursion（直接递归）
indirect recursion（间接递归）
infinite recursion 无限递归
recursive function（递归函数）
recursive helper function（递归辅助函数）
stopping condition（终止条件）
tail recursive（尾递归）

本章总结

1. *递归函数*是一个直接或间接调用它自己的函数。要终止一个递归函数，必须有一个或多个*基础情况*。
2. 递归是程序控制的另外一种可选择形式。本质上它是没有循环控制的重复。对于用其他方法很难解决而本质上是递归的问题，使用递归可以给出一个简单、清楚的解决方案。
3. 为了进行递归调用，有时候需要修改原始函数使其接收附加的参数。为达到这个目的，可以定义*递归辅助函数*。
4. 递归会产生相当大的系统开销。程序每调用一次函数，系统就必须给函数中所有的局部变量和参数分配空间。这就要消耗大量的计算机内存，并且需要额外的时间来管理这些附加的空间。
5. 如果从递归调用返回时没有待处理的操作要完成，那么这个递归的函数就被称为*尾递归*。尾递归是高效的。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

第 15.2 ～ 15.3 节

*15.1 （使用递归对一个整数的数字求和）编写一个递归函数来计算一个整数中的数字之和。使用下面函数头：

```
def sumDigits(n):
```

例如：sumDigits(234) 返回 2+3+4 = 9。编写一个测试程序提示用户输入一个整数并显示它的和。

*15.2 （斐波那契数）使用迭代改写程序清单 15-2 中的 fib 函数。（提示：不使用递归来计算 fib(n)，你首先要获取 fib(n−2) 和 fib(n−1)。）设 f0 和 f1 表示前面的两个斐波那契数，那么当前的斐波那契数就是 f0+f1。这个算法的描述如下。

```
f0 = 0 # For fibs(0)
f1 = 1 # For fib(1)

for i in range(2, n + 1):
    currentFib = f0 + f1
    f0 = f1
    f1 = currentFib

# After the loop, currentFib is fib(n)
```

编写一个测试程序提示用户输入一个序号并显示它的斐波那契数。

*15.3 （使用递归求最大公约数）求最大公约数的 gcd(m, n) 也可以如下递归地定义：

- 如果 m%n 为 0，那么 gcd(m , n) 的值为 n。
- 否则，gcd(m , n) 就是 gcd(n , m%n)。

15.4 （数列求和）编写一个递归函数来计算下面的级数。

$$m(i)=1+\frac{1}{2}+\frac{1}{3}+\cdots+\frac{1}{i}$$

编写一个测试程序显示 m（i），i = 1、2、…、10。

15.5 （数列求和）编写一个递归函数来计算下面的级数。

$$m(i)=\frac{1}{3}+\frac{2}{5}+\frac{3}{7}+\frac{4}{9}+\frac{5}{11}+\frac{6}{13}+\cdots+\frac{i}{2i+1}$$

编写一个测试程序显示 m（i），i = 1、2、…、10。

15.6 （数列求和）编写一个递归函数来计算下面的级数。

$$m(i)=\frac{1}{2}+\frac{2}{3}+\cdots+\frac{i}{i+1}$$

编写一个测试程序，程序要求用户输入一个整数 i，并显示 m（i）。

*15.7 （斐波那契数列）修改程序清单 15-2，使程序可以找出调用 fib 函数的次数。（提示：使用一个全局变量，每当调用这个函数时，该变量就加 1。）

第 15.4 节

*15.8 （以逆序输出一个整数中的数字）编写一个递归函数，使用下面的函数头在控制台上以逆序显示一个整型值。

```
def reverseDisplay(value):
```

例如：调用 reverseDisplay(12345) 显示的是 54321。编写一个测试程序，提示用户输入一个整数然后逆序输出这个整数的数字。

*15.9 （以逆序输出一个字符串中的字符）编写一个递归函数，使用下面的函数头在控制台上以逆序显示一个字符串。

```
def reverseDisplay(value):
```

例如：reverseDisplay("abcd") 显示的是 dcba。编写一个测试程序提示用户输入一个字符串然后逆序输出这个字符串。

*15.10 （字符串中某个指定字符出现的次数）编写一个递归函数，使用下面的函数头求一个指定字符在字符串中的出现次数。

```
def count(s, a):
```

例如：count("Welcome" , 'e') 返回 2。编写一个测试程序，提示用户输入一个字符串和一个字符，然后显示这个字符在这个字符串中出现的次数。

第 15.5 节

**15.11 （以逆序打印字符串中的字符）使用辅助函数改写编程题 15.9，将子串的 high 下标传递给这个函数。辅助函数头为：

```
def reverseDisplayHelper(s, high):
```

*15.12 （找出列表中的最大数）编写一个递归函数，返回一个列表中的最大数。编写一个测试程序提示用户输入一个整数列表并显示其中最大的元素。

*15.13 （求字符串中大写字母的个数）编写一个递归函数，返回一个字符串中大写字母的个数，函数头如下所示。

```
def countUppercase(s):
def countUppercaseHelper(s, high):
```

编写一个测试程序提示用户输入一个字符串并显示字符串中大写字母的个数。

*15.14 （字符串中某个指定字符出现的次数）使用一个辅助函数重写编程题 15.10，将子串的 high 下标传递给这个函数。这个辅助函数头如下所示。

```
def countHelper(s, a, high):
```

*15.15 （求列表中大写字母的个数）编写一个递归函数，返回一个字符列表中大写字母的个数。需要定义下面两个函数，第二个函数是一个递归辅助函数。

```
def count(chars):
def countHelper(chars, high):
```

编写一个测试程序，提示用户在一行输入一个字符列表并显示该列表中大写字母的个数。

*15.16 （列表中某个指定字符出现的次数）编写一个递归函数，求出列表中某个指定字符出现的次数。需要定义下面两个函数，第二个函数是一个递归辅助函数。

```
def count(chars, ch):
def countHelper(chars, ch, high):
```

编写一个测试程序提示用户在一行里输入一个字符列表和一个字符，然后显示这个列表中该字符的出现次数。

第 15.6 ～ 15.11 节

*15.17 （Tkinter：谢尔宾斯基三角形）修改程序清单 15-9，让用户使用单击鼠标左键或右键将当前阶数增 1 或减 1。初始阶数为 0。

*15.18 （汉诺塔）修改程序清单 15-8，使程序可以求得将 n 个盘子从塔 A 移到塔 B 所需的移动次数。（提示：使用一个全局变量，每当移动一次，该变量就加 1。）

*15.19 （将十进制数转换为二进制数）编写一个递归的函数，将一个十进制数转换为一个二进制数。函数头如下：

```
def decimalToBinary(value):
```

编写一个测试程序提示用户输入一个十进制数，然后显示它的二进制形式。

*15.20 （将十进制数转换为十六进制数）编写一个递归函数，将一个十进制数转换为对应的十六进制数。函数头如下：

```
def decimalToHex(value):
```

编写一个测试程序提示用户输入一个十进制数，然后显示它的十六进制形式。

*15.21（将二进制数转换为十进制数）编写一个递归函数，将一个二进制数的字符串转换为一个十进制数。函数头如下：

```
def binaryToDecimal(binaryString):
```

编写一个测试程序提示用户输入一个二进制数的字符串，然后显示它的十进制数。

15.22（将十六进制数转换为十进制数）编写一个递归函数，将一个十六进制数的字符串转换为一个十进制数。函数头如下：

```
def hexToDecimal(hexString):
```

编写一个测试程序提示用户输入一个十六进制数的字符串，然后显示它的十进制数。

**15.23（字符串排列）编写一个递归函数，输出一个字符串的所有排列。例如，对于字符串 abc，输出为：

```
abc
acb
bac
bca
cab
cba
```

（提示：定义下面两个函数，第二个函数是一个辅助函数。

```
def displayPermuation(s):
def displayPermuationHelper(s1, s2):
```

第一个函数简单地调用 dispalyPermuation(" ",s)。第二个函数使用循环，将一个字符从 s2 移到 s1，并使用新的 s1 和 s2 递归地调用该函数。基础情况是 s2 为空，将 s1 打印到控制台。）

编写一个测试程序提示用户输入一个字符串并显示它所有的排列。

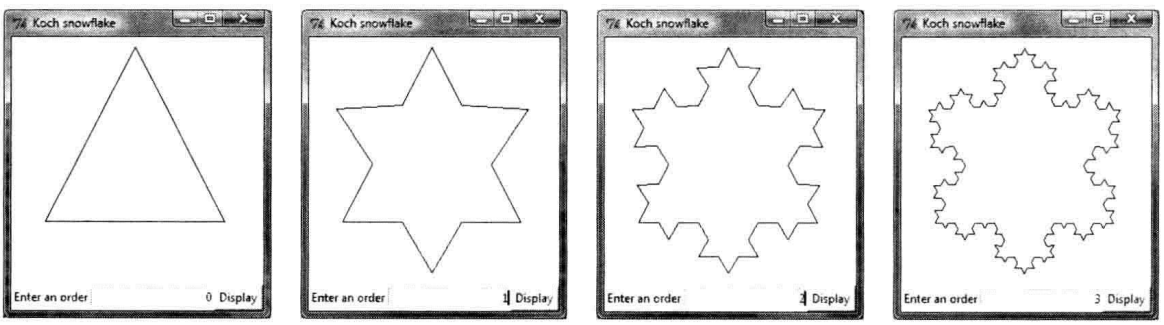
*15.24（某个目录下的文件数目）编写一个程序，提示用户输入一个目录，然后显示该目录下的文件数。

**15.25（Tkinter：科赫雪花分形）第 15.8 小节给出了谢尔宾斯基三角形分形。在本题中，编写一个程序显示另一个被称为科赫雪花的分形，它是根据一位著名的瑞典数学家的名字命名的。科赫雪花按如下方式产生：

1）从一个等边三角形开始，将其作为 0 阶（或 0 级）科赫分形，如图 15-14a 所示。

2）将图形中的每条边分成三个相等的线段，以中间的线段作为底边向外画一个等边三角形，产生 1 阶科赫分形，如图 15-14b 所示。

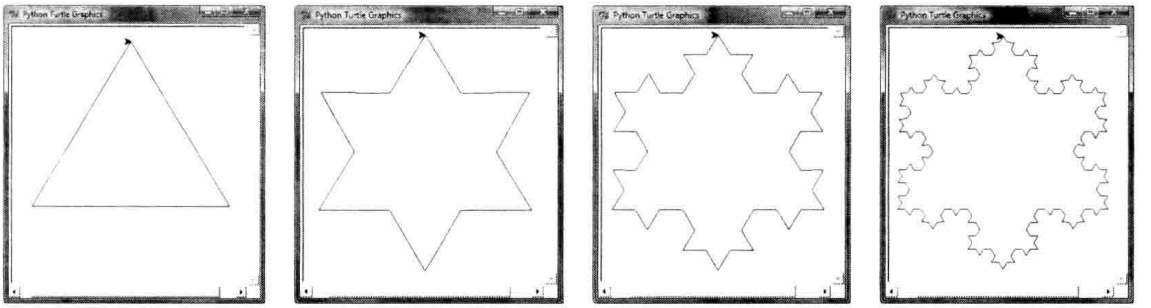
3）重复步骤 2）产生 2 阶科赫分形、3 阶科赫分形、……，如图 15-14c、图 15-14d 所示。



a）0 阶科赫分形　b）1 阶科赫分形　c）2 阶科赫分形　d）科赫雪花分形

图　15-14

**15.26（Turtle：科赫雪花分形）使用 Turtle 重写编程题 15.25 中的科赫雪花程序，如图 15-15 所示。程序提示用户输入阶数并显示对应的分形。



a）0 阶科赫分形　b）1 阶科赫分形　c）2 阶科赫分形　d）科赫雪花分形

图 15-15

**15.27 （所有的八皇后）修改程序清单 15-10，找出八皇后问题的所有可能的解决方案。

**15.28 （找出单词）编写一个程序，递归地找出某个目录下的所有文件中某个单词出现的次数。程序应当提示用户输入一个目录名。

**15.29 （Tkinter：H 树分形）一个 H 树分形定义如下：

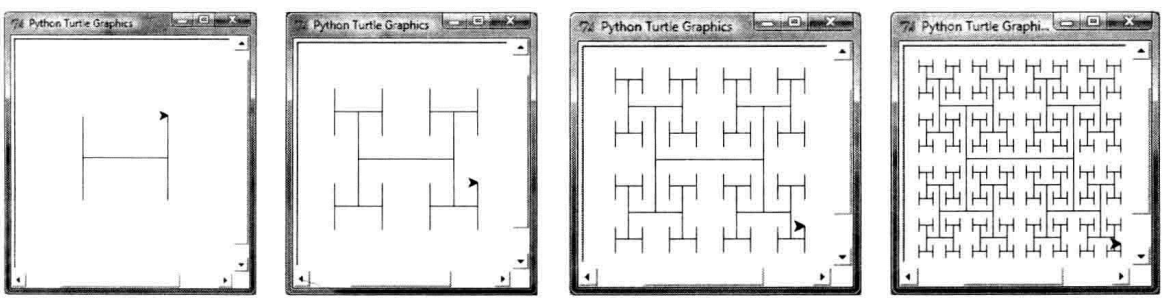
1）从字母 H 开始。H 的三条线长度一样，如图 15-1a 所示。

2）字母 H（以它的 sans-serif 形式，H）有四个端点。以这四个端点为中心位置绘制一个 1 阶 H 树，如图 15-1b 所示。这些 H 的大小是包括这四个端点的 H 的一半。

3）重复步骤 2）来创建 2 阶、3 阶、……H 树，如图 15-1c、15-1d 所示。

编写一个绘制 H 树的 Python 程序，如图 15-1 所示。

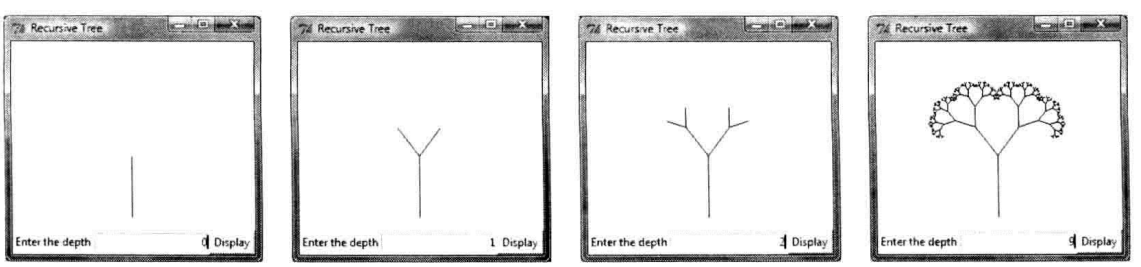
**15.30 （Turtle：H 树分形）使用 Turtle 重写编程题 15.29 中的 H 树分形程序，如图 15-16 所示。程序提示用户输入阶数并显示相应阶数的分形。



a）0 阶 H 树　b）1 阶 H 树　c）2 阶 H 树　d）3 阶 H 树

图 15-16

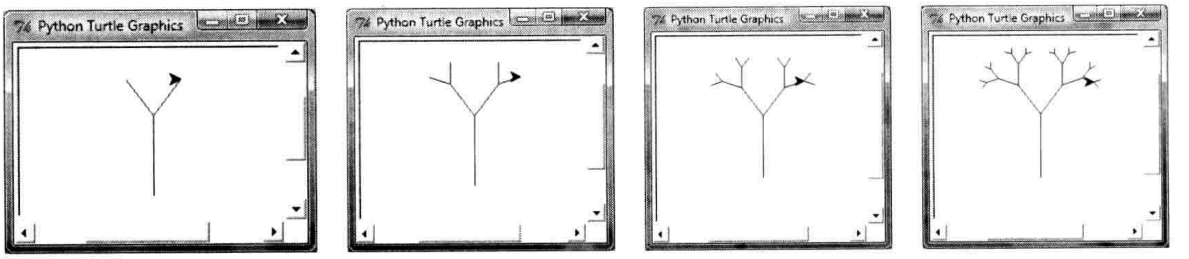
**15.31 （Tkinter：递归树）编写一个程序来显示一个递归树，如图 15-17 所示。



a）0 阶递归树　b）1 阶递归树　c）2 阶递归树　d）9 阶递归树

图 15-17 一个带特定深度的递归树

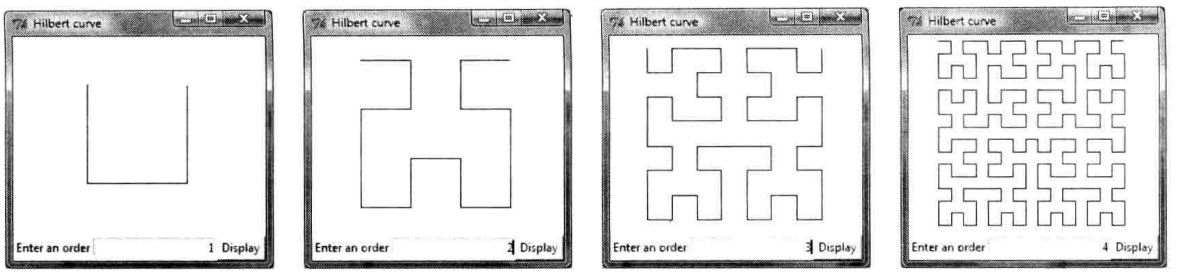
**15.32 （Turtle：递归树）使用 Turtle 重写编程题 15.31 中的程序，如图 15-18 所示。程序提示用户输入阶数并显示相应阶数的分形。



a）0 阶递归树　b）1 阶递归树　c）2 阶递归树　d）3 阶递归树

图　15-18

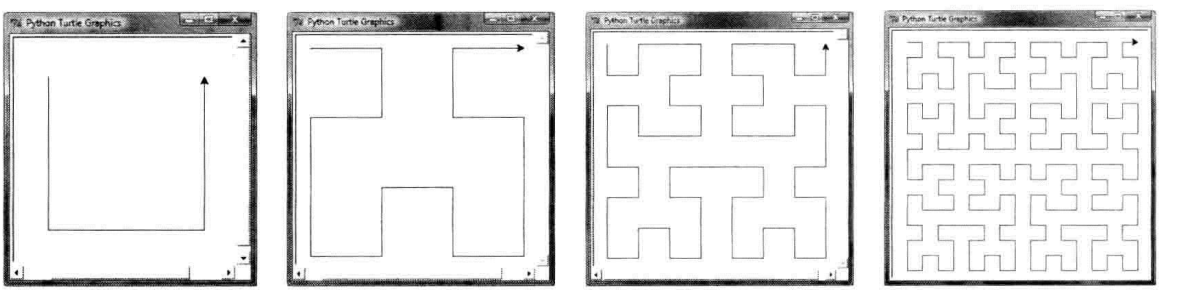
**15.33 （Tkinter：希尔伯特曲线）希尔伯特曲线，是由德国数学家 David Hilbert 于 1891 年首先提出的，它是一个访问一个正方网格所有点的空间填充曲线，这个网格的大小可以是 2×2、4×4、8×8、16×16 或其他 2 的幂次。编写一个程序显示一个指定阶数的希尔伯特曲线，如图 15-19 所示。



a）1 阶希尔伯特曲线　b）2 阶希尔伯特曲线　c）3 阶希尔伯特曲线　d）4 阶希尔伯特曲线

图　15-19

**15.34 （Turtle：希尔伯特曲线）使用 Turtle 重写编程题 15.33 中的希尔伯特曲线，如图 15-20 所示。程序提示用户输入阶数并显示相应阶数的分形。



a）0 阶希尔伯特曲线　b）1 阶希尔伯特曲线　c）2 阶希尔伯特曲线　d）3 阶希尔伯特曲线

图　15-20

15.35 （Tkinter：谢尔宾斯基三角形）修改程序清单 15-9，显示填充的谢尔宾斯基三角形，如图 15-21 所示。

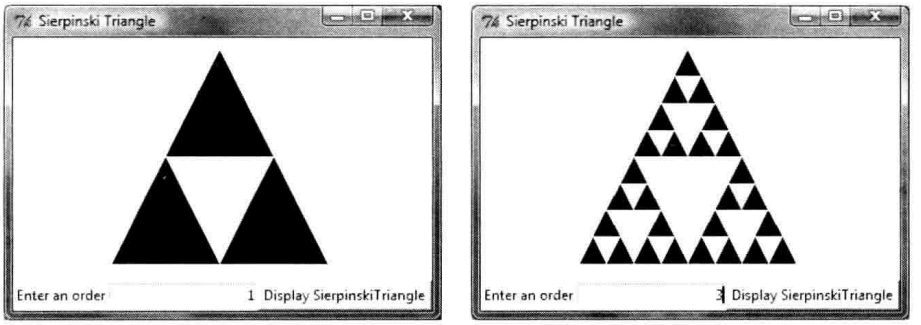


图 15-21 显示被填充的谢尔宾斯基三角形

15.36 （Turtle：谢尔宾斯基三角形）使用 Turtle 重写程序清单 15-9。

Python 关键字

Python 语言保留下面的关键字。它们不应该被用在 Python 预定义的目的之外的其他任何地方。

and	else	in	return
as	except	is	True
assert	False	lambda	try
break	finally	None	while
class	for	nonlocal	with
continue	from	not	yield
def	global	or	
del	if	pass	
elif	import	raise	

ASCII 字符集

表 B-1 和表 B-2 给出 ASCII 字符以及它们各自的十进制和十六进制码。一个字符的十进制码或十六进制码是它的行索引和列索引的组合。例如：在表 B-1 中，字母 A 在第 6 行第 5 列，所以它对应的十进制数是 65；在表 B-2 中，字母 A 在第 4 行第 1 列，所以它等价的十六进制数是 41。

表 B-1　十进制索引表示的 ASCII 码字符集

	0	1	2	3	4	5	6	7	8	9
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
1	nl	vt	ff	cr	so	si	dle	dcl	dc2	dc3
2	dc4	nak	syn	etb	can	em	sub	esc	fs	gs
3	rs	us	sp	!	"	#	$	%	&	'
4	(	)	*	+	,	–	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~	del		

表 B-2　十六进制索引表示的 ASCII 码字符集

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht	nl	vt	ff	cr	so	si
1	dle	dcl	dc2	dc3	dc4	nak	syn	etb	can	em	sub	esc	fs	gs	rs	us
2	sp	!	"	#	$	%	&	'	(	)	*	+	,	–	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	del

数制系统

C.1 简介

计算机内部使用二进制数，因为计算机很自然地就是存储和处理 0 和 1 的。二进制数系有两个数字：0 和 1。数字和字符都被存储为由 0 和 1 组成的序列。每个 0 或 1 都被称为一个比特（二进制数）。

在日常生活中，我们使用的是十进制数。当在程序中写出一个像 20 这样的数字时，它被认为是一个十进制数。计算机内部会使用软件将十进制数都转换成二进制数，反之亦然。

我们使用十进制数编写计算机程序。但是为了便于操作系统处理它们，我们需要到达使用二进制数的“机器层”。二进制数通常都很长而且很繁琐。经常会使用十六进制数来简化它们，每个十六进制数代表四个二进制数。十六进制数系有 16 个数字：0 ～ 9 和 A ～ F。字母 A、B、C、D、E 和 F 分别对应十进制数 10、11、12、13、14 和 15。

十进制数系中的数字是 0、1、2、3、4、5、6、7、8 和 9。一个十进制数是用由一个或多个这样的数字组成的序列来表示的。每个数字所代表的值依赖于它所处的位置，它的位置表示 10 的整数幂。例如：十进制数 7423 中的数字 7、4、2 和 3 分别表示 7000、400、20 和 3，如下所示。

7	4	2	3

$$= 7 \times 10^3 + 4 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$$

$$10^3 10^2 10^1 10^0 = 7000 + 400 + 20 + 3 = 7423$$

十进制数系有十个数字，而它的位置值是 10 的整数幂。我们可以说 10 是十进制数系的基数。同样，由于二进制数系有两个数字，那它的基数是 2，而十六进制数系有 16 个数字，那它的基数是 16。

如果 1101 是一个二进制数，那么数字 1、1、0 和 1 分别表示 1×2^3、1×2^2、0×2^1 和 1×2^0。

1	1	0	1

$$= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$2^3 2^2 2^1 2^0 = 8 + 4 + 0 + 1 = 13$$

如果 7423 是一个十六进制数，那么 7、4、2 和 3 分别表示 7×16^3，4×16^2，2×16^1 和 3×16^0。

7	4	2	3

$$= 7 \times 16^3 + 4 \times 16^2 + 2 \times 16^1 + 3 \times 16^0$$

$$16^3 16^2 16^1 16^0 = 28\,672 + 1024 + 32 + 3 = 29\,731$$

C.2 二进制和十进制数之间的转换

假设有一个二进制数 $b_n b_{n-1} b_{n-2} \cdots b_2 b_1 b_0$，那么它等价的十进制值是

$$b_n \times 2^n + b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \ldots + b_2 \times 2^2 + b_1 \times 2^1 + b_0 \times 2^0$$

下面是一些将二进制数转换为十进制的例子：

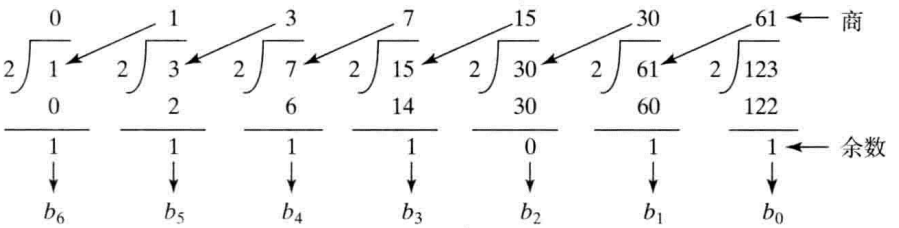
二进制数	转换公式	十进制数
10	$1 \times 2^1 + 0 \times 2^0$	2
1000	$1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$	8
10101011	$1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$	171

将十进制数 d 转换为二进制数就是找出比特 $b_n, b_{n-1}, b_{n-2}, \cdots, b_2, b_1$ 和 b_0 满足

$$d = b_n \times 2^n + b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \ldots + b_2 \times 2^2 + b_1 \times 2^1 + b_0 \times 2^0$$

这些比特可以通过 d 不断地除以 2，直到商为 0 来找出。余数就是 b_0，b_1，$b2$，$\cdots$ b_{n-2}，b_{n-1} 和 b_n。

例如：十进制数 123 是二进制数 1111011。转换过程如下所示。



提示：Windows 中的计算器，如图 C-1 所示，是实现数字转换的一个很有用的工具。为了运行它，在开始按钮中搜索到“Calculator”，然后单击“View”→“Scientific”按钮。

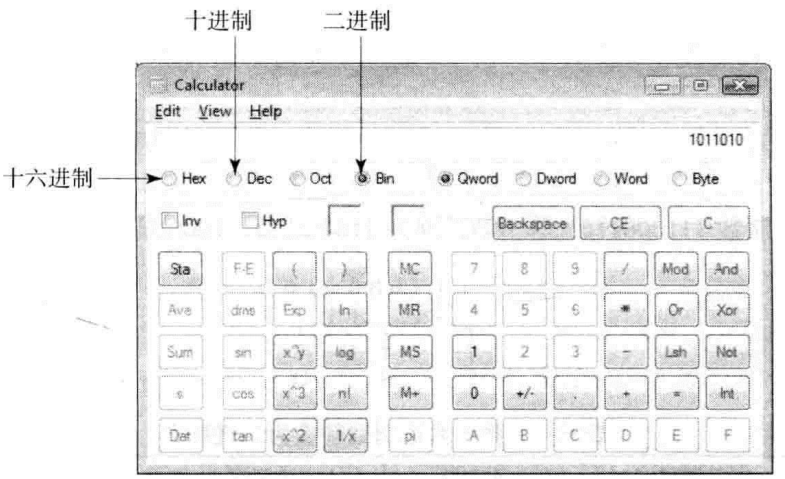


图 C-1 你可以使用 Windows 的计算器实现数制转换

C.3 十六进制和十进制数之间的转换

假设有一个十六进制数 $h_n h_{n-1} h_{n-2} \cdots h_2 h_1 h_0$，它等价的十进制值就是

$$h_n \times 16^n + h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \ldots + h_2 \times 16^2 + h_1 \times 16^1 + h_0 \times 16^0$$

下面是一些将十六进制数转换为十进制数的例子。

十六进制数	转换公式	十进制数
7F	$7 \times 16^1 + 15 \times 16^0$	127

（续）

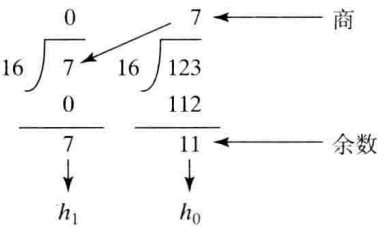
十六进制数	转换公式	十进制数
FFFF	$15\times16^3+15\times16^2+15\times16^1+15\times16^0$	65 535
431	$4\times16^2+3\times16^1+1\times16^0$	1073

为了将一个十进制数 d 转换成一个十六进制数，就是要找到 h_n，h_{n-1}，h_{n-2}，$\cdots h_2$，h_1 和 h_0 满足

$$d=h_n\times16^n+h_{n-1}\times16^{n-1}+h_{n-2}\times16^{n-2}+\ldots+h_2\times16^2+h_1\times16^1+h_0\times16^0$$

这些数字可以通过将 d 不断地除以 16 直到商为零。依次得到的余数就是 h_0，h_1，h_2，$\cdots$ h_{n-2}，h_{n-1} 和 h_n。

例如：十进制数 123 就是十六进制数 7B。转换如下所示：



C.4　二进制数和十六进制数之间的转换

为了将一个十六进制数转换为一个二进制数，只要使用表 C-1 就可以将十六进制数中的每个数字转为一个四位的二进制数。

例如：十六进制数 7B 是 1111011，这里的 7 用二进制表示为 111，而 B 用二进制表示为 1011。

为了将一个二进制数转换为一个十六进制数，将二进制数从右向左每四个数转换为一个十六进制数。

例如：二进制数 1110001101 是 38D，因为 1101 是 D，1000 是 8，而 11 是 3，如下所示。

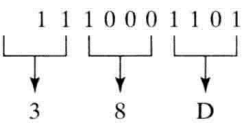


表 C-1　将十六进制数转换为二进制数

十六进制数	二进制数	十进制数	十六进制数	二进制数	十进制数
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

注意：八进制数也非常有用。八进制数系有 8 个数字：0 到 7。十进制数 8 在八进制数

系中用 10 表示。

下面是练习数制转换的一些很好的在线资源。

- http://forums.cisco.com/CertCom/game/binary_game_page.htm
- http://people.sinclair.edu/nickreeder/Flash/binDec.htm
- http://people.sinclair.edu/nickreeder/Flash/binHex.htm

复习题

1. 将下面的十进制数转为十六进制数和二进制数。

```
100; 4340; 2000
```

2. 将下面的二进制数转换为十六进制数和十进制数。

```
1000011001; 100000000; 100111
```

3. 将下面的十六进制数转换为二进制数和十进制数。

```
FEFA9; 93; 2000
```